



# COMPILER DESIGN

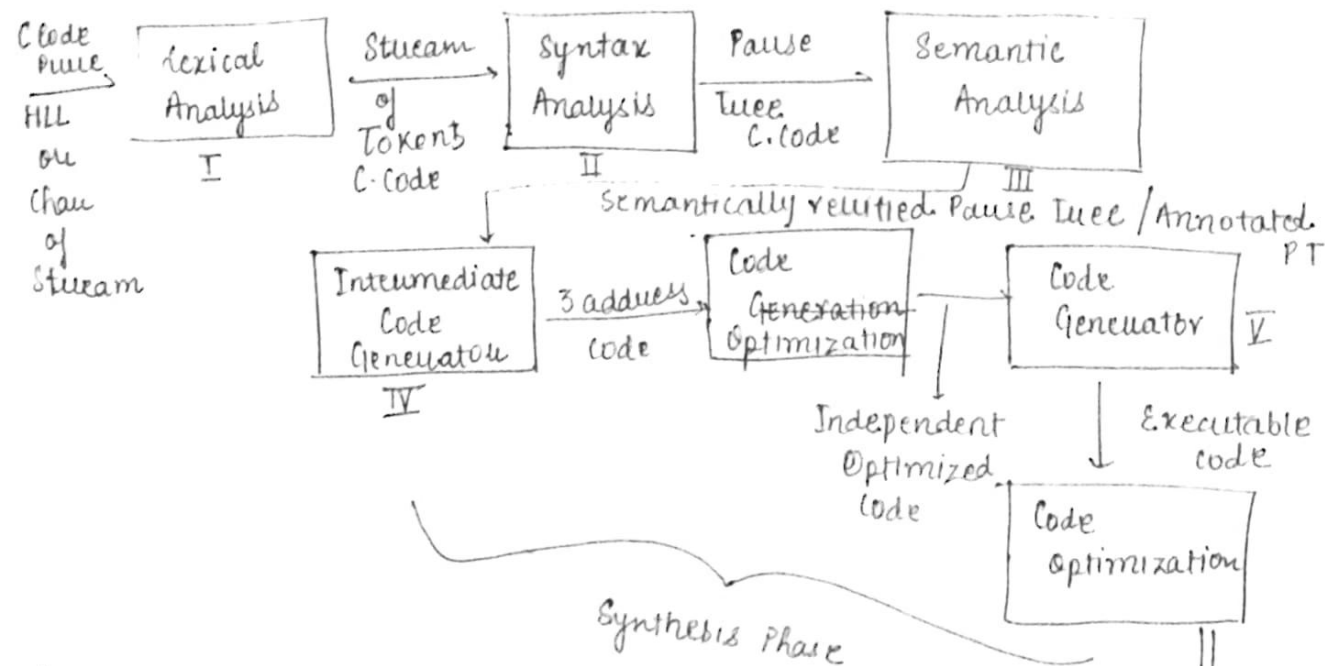


*Handwritten Notes!*

# Lecture - 01

## Phases of Compiler :-

### Analysis phases



First 3 Steps - Analysis phase

Last 4 are - Synthesis phase

Total Phases  
5 out

Code Optimization is not a crucial part of phases

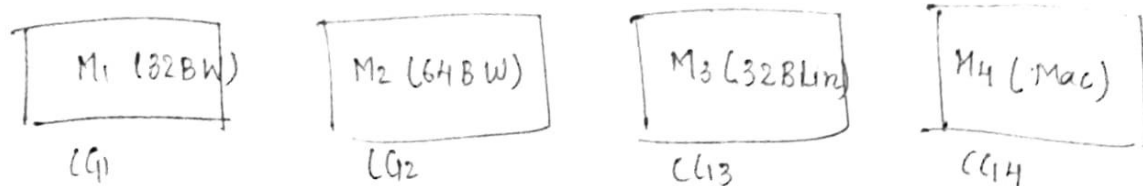
Utl ICG (IV) . Compiler generates Independent Code. } Utl ICG we have front end.  
 & Code Generator generate machine dependent Code. } Comes under backend.

## Why 7 Phases ?

If same compiler is to be applied to different machines of diff architecture

Frontend or Independent Code Generation part will remain same but the Code Generator will get changed according to the Architecture of Machine.

In General for the below machines we need 4 x 5 = 20 phases



if we want to generate compiler we need only 4 Independent +  
 4 CGs → Total 8 phases

2) If I want different compilers on diff. languages for same machine for eg - C, C++, Java, python

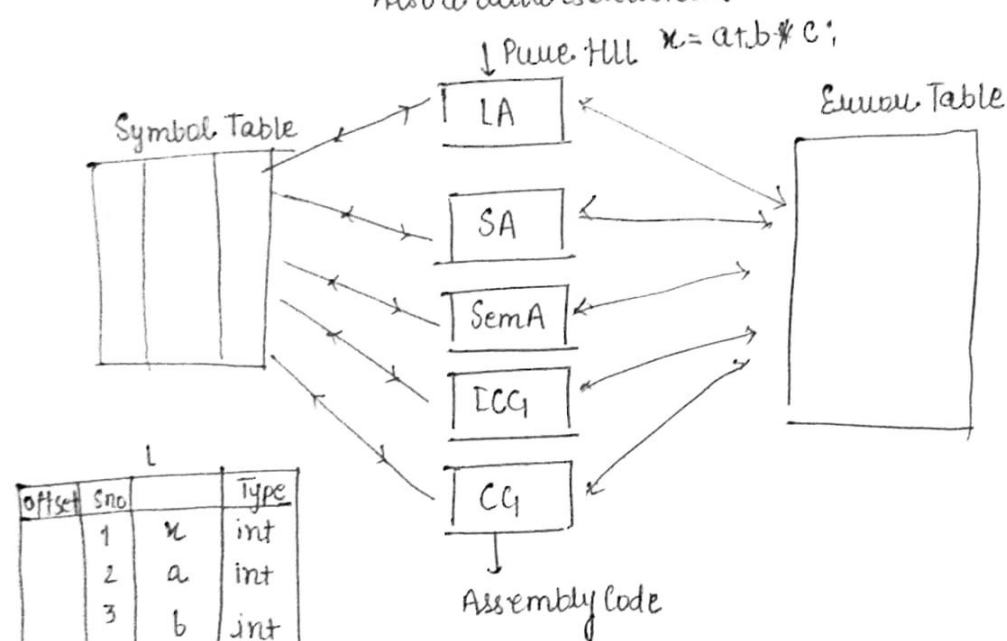
We need -  $4 \times 4 + 1 = 17$  phases

4 languages  $\times$  4 phases each (Independent Code phases)

Symbol Table :  $\rightarrow$  Stores the Identifiers and Constants  
- It is a data structure.

Symbol Handler Table :  $\rightarrow$  Store the Symbols present in code.

- Also a data structure



| Offset | Sno |   | Type |
|--------|-----|---|------|
|        | 1   | x | int  |
|        | 2   | a | int  |
|        | 3   | b | int  |

$\rightarrow$  data structure

$\rightarrow$  Operation in ST

- $\rightarrow$  Insert
- $\rightarrow$  Search
- $\rightarrow$  Delete
- $\rightarrow$  Modify

List  
Tree

HashTable -  $O(1)$   
# Mostly used

For eg  $\rightarrow$   $S \rightarrow id = E;$   
 $E \rightarrow E + T / T$   
 $T \rightarrow T * F / F$   
 $F \rightarrow id$

Now generating parse Tree.

$x = a + b * c;$

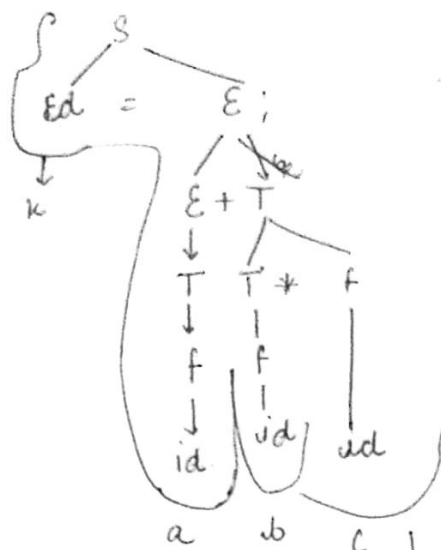
$\downarrow$   
LA will scan it

$id_1, op, id_2, op, id_3, op, id_4, sp$

Stream of Tokens

Syntax Analyzer will check whether the stream of tokens can be generated by the CFG

Here,



∴ if in any case we can generate parse tree for the context free grammar that matches the stream of tokens, this means the code is syntactically correct

This will move to Semantic Analyzer.

Semantic Analyzer

↓  
Semantically Verified Parse Tree →

- It will check
  - Type checking
  - Type conversion
  - Variable declaration.

move to

ICG → will generate 3 address Code

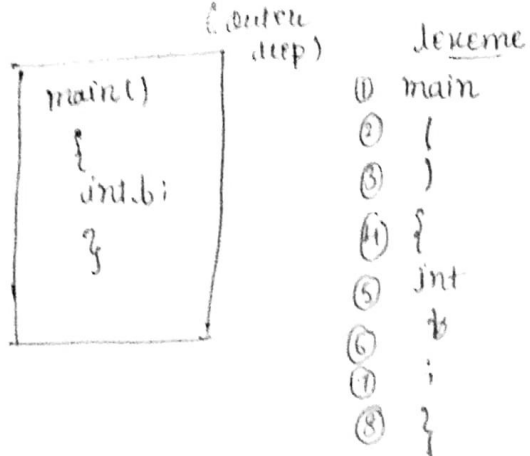
Eg  $\begin{aligned} &= \text{Compute} \\ &t_1 = b * c; \\ &t_2 = t_1 + a; \\ &\kappa = t_2; \end{aligned} \quad \left. \vphantom{\begin{aligned} &t_1 = b * c; \\ &t_2 = t_1 + a; \\ &\kappa = t_2; \end{aligned}} \right\} \text{Three add code.}$

↓  
C: Optimization } machine independent  
↓  
 $\begin{aligned} &t_1 = b * c; \\ &\kappa = t_1 + a; \end{aligned}$

Lexical Analyzer



Lexical Analyzer is a program that recognize 'lexemes' in the code and produce stream of tokens as a output and it also produce lexical error if any.



Token (internal representation  
of compiler)

id

Special Symbol / Operator token

" "

Special Symbol

Keyword

identifier

Special Symbol

Special Symbol

lexeme - Smallest unit of a given input

lexeme  $\xrightarrow{\text{forms}}$  Token

Tokens  $\rightarrow$  Identifier - letter, digit (but not starting with digit)

$\rightarrow (L+_) (L+d+_)^*$

- ↳ Keyword - 32 type of Keyword in C
- ↳ Constant  $\rightarrow$  'A', 'Gate21'
- ↳ Special Symbol  $\rightarrow$  (, ), {, }
- ↳ Operator  $\rightarrow$  +, -, \*, /

functionality of LA  $\rightarrow$  ~~Start from~~ 1. It recognise lexeme and produce stream of tokens.

2. It recognise white spaces, tab, comment and ignore or delete it.
3. It gives lexical error if any and also produce line no.
4. It scan start from beg of program and goes to end.
5. It matches the longest prefix.

$\hookrightarrow$  gate;  
 $T_g$  is an identifier but not a token  
 full gate is a token

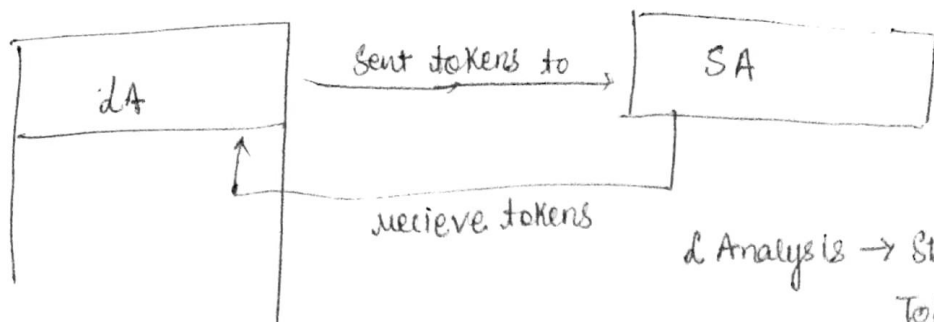
Loop Holes in LA

$x?y;z;$  5 tokens ? will be taken as single

$\hookrightarrow$  LA will ask for Syntax analyzer to get no of tokens

$x+=y++;$  5 tokens  
 $\hookrightarrow$  it is a single token

char \*p = 'gate2021';  
 $\hookrightarrow$  This will be taken as single one.



A [50];

[ ] will be a single token  
 $\hookrightarrow$  4 tokens

sizeof(x);

$\hookrightarrow$  3 tokens

int x /\* comment \*/ ;  $\rightarrow$  5 tokens

int x ; comment /\*  $\rightarrow$  6 tokens

Build a Lexical Analyzer

$\Downarrow$

Pattern matching system

$(d+-)(d+-+d)^+$

using RE  $\rightarrow$  RL  $\rightarrow$  FA

KISCO

• Keyword • Special symbol • operator  
 • Identifier • constant

Ambiguous and Non-Ambiguous grammar.

Afterwards

Symbol table

$\hookrightarrow$  list  
 $\hookrightarrow$  Hashing  
 $\hookrightarrow$  Binary tree.

y + z = x;  $\rightarrow$  6 tokens

no lexical error

Syntax error  $\rightarrow$  can't generate parse tree.

int labc;  $\rightarrow$  lexical error

can't generate token

int float;  $\rightarrow$  No lexical error - 3 tokens

Syntax error

int x = 10;

int y = 20;  $\rightarrow$  No lexical error - 6 tokens

z = x + y;  $\rightarrow$  Semantical error z was not declared.

Grammar

$\{V, T, P, S\}$

$\hookrightarrow$  Variable / non-terminal

$\hookrightarrow$  Terminal

$\hookrightarrow$  Production

$\hookrightarrow$  start symbol

eg  $\rightarrow$   $E \rightarrow E + E$

$E \rightarrow E * E$

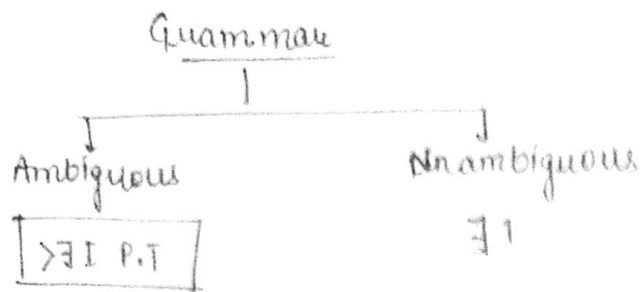
$E \rightarrow E \uparrow E$

$E \rightarrow id$

$V = \{E\}$

$S \rightarrow E$

Terminal =  $\{+, *, \uparrow, id\}$



$\exists$  more than one derivation / p, T  
for any word  $\in L(G)$

$\exists$  only one derivation / p, T  
for all word.

for grammar to be ambiguous grammar must have left and Right Recursion

for Eg  $\Rightarrow S \rightarrow SS/a$

$w = a$

$w = aa$

$w = aaa$

ambiguous

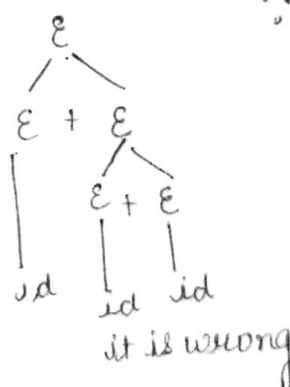
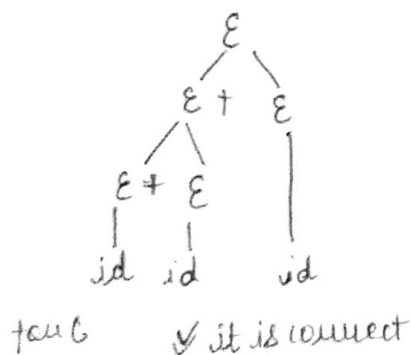


$S \rightarrow SAS \mid SBS \mid \epsilon$        $S \rightarrow SSS/a$

$E \rightarrow E + E / id$

yes ambiguous

for  $w = id + id + id$



$\therefore$  in C we have  
+ is Left to Right  
Associative.

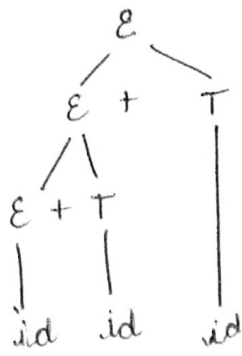
$E \rightarrow E + E / id$

$\Downarrow$

unambiguous grammar.

$E \rightarrow E + T / T$

$T \rightarrow id$



$E \rightarrow E + T / T$   
 $T \rightarrow id$   
 (we can write it as)  
 $E \rightarrow E + id / id$

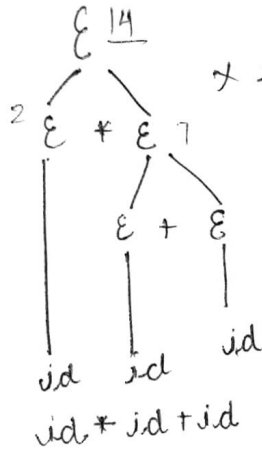
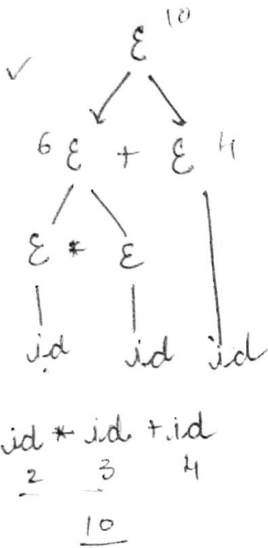
if the grammar is left recursive it is left associative

we can convert some ambiguous grammar to unambiguous

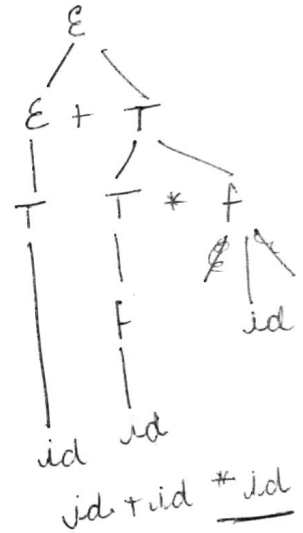
Not all

$E \rightarrow E + E / E * E / id$

$\xrightarrow{\text{unambiguous}} E \rightarrow E + T / T$   
 $T \rightarrow T * f / f$



$f \rightarrow id$



$E \rightarrow E + E / E * E / E \uparrow E / id$

$\Downarrow$   
 $E \rightarrow E + T / T$

$T \rightarrow T * f / f$

$f \rightarrow f \uparrow D / D$

$D \rightarrow id$

$\uparrow > * > +$

$\uparrow$  is right associative

$*_L > *_R$

$\uparrow_L < \uparrow_R$

Acceft

There is no algorithm that would find out whether given grammar is ambiguous.

Ambiguous  $\rightarrow$  2 parse trees  $\rightarrow$  2 LMD or 2 RMD

Left Most

Derivation

(if Both side derivation)  $\xleftarrow{\text{meaning}}$  if LMD = RMD the grammar is unambiguous.  
 give same ans

## Lecture 3/19

$$A \rightarrow A @ B / B$$

$$B \rightarrow B \# C / C$$

$$C \rightarrow C \$ D / D$$

$$D \rightarrow id$$

fasten the symbol precedence

$$[\$ > \# > @]$$

We can't convert all ambiguous  $\$L > \$R$

grammar to unambiguous  $\#L > \#R$

undecidable problem.  $@L > @R$

## Lecture

due 5

$$c? (b \text{ or } a)^* c$$

$$T_3 T_3$$

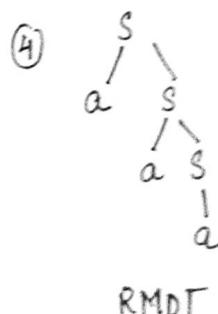
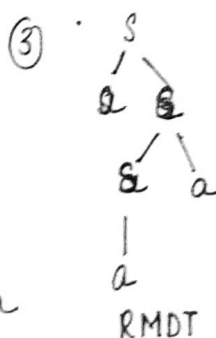
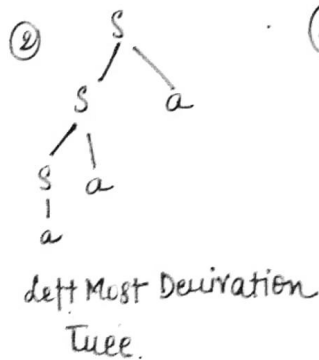
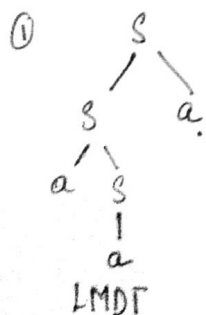
$$(C^*)$$

$$\frac{bbaac}{T_3} \frac{abc}{T_3}$$

## Lecture 4/19

Ambiguous and Unambiguous

$$S \rightarrow Sa / aS / a$$



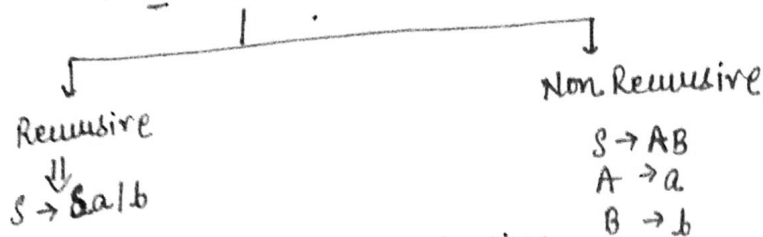
→ Hence we have 2 LMDT and 2 RMDT and grammar is

Ambiguous

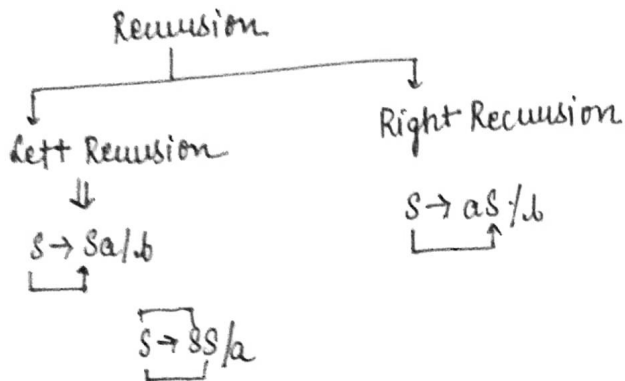
→ The ambiguous grammar from which we cannot eliminate ambiguity is called inherently ambiguous grammar.

→ For Unambiguous Grammar LMDT = RMDT

Variable  $\rightarrow$  (Variable + Terminal)<sup>\*</sup>



There exist at least one production having same variable in LHS & RHS



Both Left and Right

$A \rightarrow A\alpha/B \Rightarrow$  Top Down Parser / RDP

### Left Recursion

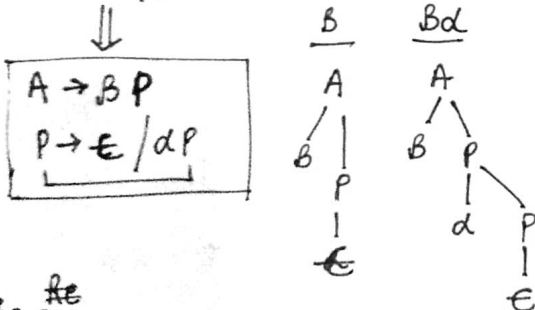
↳ it will generate

$$B, Bd, Bdd \dots Bd^*$$

Remissive  
Descent  
Pause

To make it pass through Top Down parser we need to convert it into Right

## Recursive Grammar.

$$A \rightarrow Ad/B$$


Eq-2  $A \rightarrow A\alpha / \beta_1 / \beta_2 / \beta_3 \dots \beta_n$

$$A \xrightarrow{\downarrow} B_1 P / B_2 P / \dots B_n P$$
$$p \rightarrow e/dp$$
$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow a \\ B &\rightarrow b \end{aligned}$$

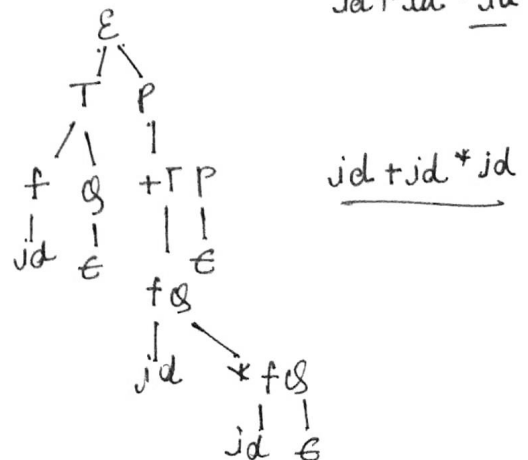
Eg - 3

$$A \rightarrow Ad_1 / Ad_2 / Ad_3 \dots - Ad_n / B$$
$$A \rightarrow BP$$
$$P \xrightarrow{\epsilon} d_1 P / d_2 P / d_3 P \dots d_n P$$
Eq-4
$$A \rightarrow Ad_1 / Ad_2 \dots Ad_n / B_1 / B_2 \dots B_n$$
$$A \rightarrow \beta_1 P / \beta_2 P \dots / \beta_n P$$
$$P \rightarrow -e / \alpha_1 P / \alpha_2 P / \alpha_3 P \dots / \alpha_n P$$

Eg-5

$$\varepsilon \rightarrow \varepsilon + T/T$$
$$\frac{1}{T} \rightarrow T^* f/f$$
$$f \rightarrow \text{id}$$

11

$$\mathcal{E} \rightarrow TP$$
$$P \rightarrow E/TP$$
$$T \rightarrow f \text{ } \mathcal{G}$$
$$Q \rightarrow E / * f Q$$
$$f \rightarrow jd$$

$$\underline{jd + jd * jd}$$

Que 2  $A \rightarrow Aab/c.$

$$\begin{aligned} A &\rightarrow CT \\ T &\rightarrow \epsilon / abT \end{aligned}$$

Ani

Que 3  $S \rightarrow (L)/a$   
 $L \rightarrow L, S/S$

Ani

$$\begin{aligned} S &\rightarrow (L)/a \\ L &\rightarrow SP \\ P &\rightarrow \epsilon / , SP \end{aligned}$$

Que 4  $A \rightarrow A(A)/a$   
 $\downarrow$   
 $A \rightarrow aT$   
 $T \rightarrow \epsilon / (A)T$

Que 5  $S \rightarrow SSS/a$   
 $\downarrow$   
 $S \rightarrow as'$   
 $s' \rightarrow \epsilon / SSS'$

Que 6  $S \rightarrow A$   
 $A \rightarrow Ad/Ae/aB/ac$   
 $B \rightarrow bBC/f$   
 $\downarrow$   
 $S \rightarrow A$   
 $A \rightarrow aBP/acP$   
 $P \rightarrow \epsilon / dP / eP$   
 $B \rightarrow bBC / f$

Que 7

$$\begin{aligned} S &\rightarrow AaB \\ A &\rightarrow aA / ABa/ba \\ B &\rightarrow AB/b \end{aligned}$$

indirect Recursion

$\downarrow$

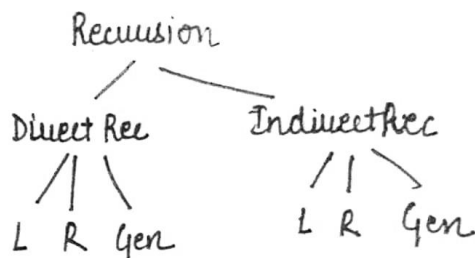
$$\begin{aligned} S &\rightarrow AaB \\ A &\rightarrow aAA' / baA' \\ A' &\rightarrow \epsilon / BaA' \\ B &\rightarrow AB/b \end{aligned}$$

Que 8  $R \rightarrow RR / Ra / aR / b$   
 $\downarrow$   
 $R \rightarrow bT / aRT$   
 $T \rightarrow \epsilon / RT / aT$

### Lecture 5

General Recursion - Neither left Recursion nor Right

$S \rightarrow aSb/b.$



Que 1

|                                                                                                                                                           |               |                                                                                                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $\begin{aligned} S &\rightarrow aBDh \\ B &\rightarrow Bb/h \\ D &\rightarrow EF \\ E &\rightarrow g/\epsilon \\ f &\rightarrow f/\epsilon \end{aligned}$ | $\Rightarrow$ | $\begin{aligned} S &\rightarrow aBDh \\ B &\rightarrow hT \\ T &\rightarrow \epsilon / bT \\ D &\rightarrow \epsilon f \\ E &\rightarrow g/\epsilon \\ f &\rightarrow f/\epsilon \end{aligned}$ |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Que 2  $E \rightarrow E + E / E * E / id$   
 $\downarrow$   
 $E \rightarrow id T$   
 $T \rightarrow \epsilon / +ET / *ET$

Que 3  $A \rightarrow AaAb / AbAa / \epsilon$  ★★

$\downarrow$

$$\begin{aligned} A &\rightarrow T \\ T &\rightarrow \epsilon / aAbT / bAaT \end{aligned}$$

$\epsilon T \Rightarrow T$

Que 4  $S \rightarrow Aa/b$   
 $A \rightarrow Ab/bc/\epsilon$   
 $\downarrow$

$$S \rightarrow Aa/b$$

~~$$A \rightarrow \epsilon/bT/T$$~~

$$A \rightarrow Ab/Aac/bc/\epsilon$$

↓

$$S \rightarrow Aa/b$$

$$A \rightarrow T/bcT$$

$$T \rightarrow \epsilon/bT/act$$

Due 5

$$A \rightarrow Aa/aA/b$$

↓

$$A \rightarrow aAT/bT$$

$$T \rightarrow \epsilon/aT$$

Due 6

$$L = \{\epsilon, a, b, c, \dots, z\}$$

\*\*\* select set which rep all 4 letter word

a)  $L^1 + L^1 + L^1$  c)  $L^4 - L^3$

b)  $L^2 + L^2$  d)  $L^4$

$L^0$  represent all  $\epsilon$

$L^1$  " " one letter word

$L^2$  " " two " "

$L^3$  " " three " "

$L^4$  " " four " "

$L^4 - L^3$  will give words with

4 letter only

Due 7 two binary operators

\*\*\*  $\uparrow > \downarrow$  (precedence)

$$a \uparrow b = a^b$$

$$a \downarrow b = \log_b a$$

$\uparrow$  = Right Associative

$\downarrow$  = left "

compute  $65536 \downarrow 2 \uparrow 14 \uparrow 2 \downarrow 2$

1.  $65536 \downarrow 2 \uparrow 14 \uparrow 2 \downarrow 2$

$$\begin{array}{r} 8 \overline{) 65536} \\ 8 \overline{) 8192} \end{array}$$

$$\begin{array}{r} 8 \overline{) 1024} \end{array}$$

$$\begin{array}{r} 8 \overline{) 128} \end{array}$$

$$\begin{array}{r} 10 \text{ Ans} \\ 2 \overline{) 16} \\ 2 \overline{) 24} \quad 16 \overline{) 4} \\ 2 \overline{) 2} \end{array}$$

| <u>due</u> | <u>operators</u> | <u>Precedence</u> | <u>Association</u> |
|------------|------------------|-------------------|--------------------|
|            | *                | 3                 | Left               |
|            | -                | 4                 | Left               |
|            | +                | 2                 | Right              |
|            | ↑                | 1                 | Right              |

greater no greater precedence

$$10 - 6 - 2 \uparrow 8 - 6 \uparrow 1 * 2 \uparrow 1 + 1$$

$$\Rightarrow 2 \uparrow 8 + 1 * 2 \uparrow 1 + 1$$

$$\Rightarrow 2 \uparrow 2 + 2 \uparrow 1 + 1$$

$$\Rightarrow 2 \uparrow 2 \uparrow 2 \uparrow 2$$

$$\Rightarrow \underline{\underline{65536}}$$

Due no of Tokens

Switch (input value) I

```

{
  case 1.: b = c * d; break;
  Default.: b = b++; break;
}

```

26

II main()

```

{
  int a, b;
  a = 10;
  b = 15;
  printf("a = %d, b = %d", a++, b--);
}

```

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28

29

Que

= Count no of Tokens

① printf ("the no of tokens are %d",  
8+count);

8

② printf ("A %B=", &i); 8

③ float abs\_Zero\_Kelvin = -273;

6

④  $x = x + (a + b) * 5;$

12

⑤  $abcd + (3 - 5 + x * 9 / 3) + 8;$

14

Que Output of Lexical analyzer

~~a) Set of Reg express~~ d) Set of token

Que What a functionality of C compiler

a) Identifying Token

b) Identifying Syntax error

c) Linking

d) Loading

Que Who check token arrangement

Syntax Analyzer

token generation is done by

Lexical Analyzer

Token

→ String of character which logically belong together

→ A pair consisting of a token name and a optional attribute value.

Lexeme

→ The sequence of character matched by a pattern to form corresponding token

Pattern

It is a description of the form that the lexeme of a token may take.

$x = a + b + c;$

| lexeme | Token |
|--------|-------|
| x      | id    |
| =      | spsy  |
| a      | id    |
| +      | op    |
| b      | id    |
| +      | op    |
| c      | id    |
| ;      | spsy  |

# 6th Lect

Parse

Top Down Parse

Bottom Up Parse (Shift Reduce Parse)

With Backtrack

Without Backtrack

(Predictive Parse)

1. UA 2. NLR

RDP (program)

LL(1)

for LL we need grammar to be left factored

OPP Operator Precedence Parse

(only parse can pass both Ambiguous & Unambiguous Grammar)

unambiguous

LR Parse

LR(0)

SLR(1)

CLR(1)

Brute force Method

Recursive Descent Parse (general)

Top Down Parse

→ Pass Unambiguous Grammar

→ no left Recursion

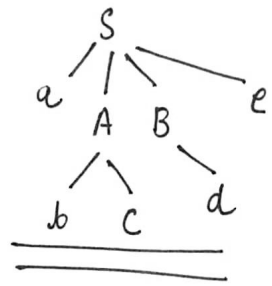
How TDP work?

$S \rightarrow aABe$

$A \rightarrow bc/b$

$B \rightarrow d$

$w = abcde$

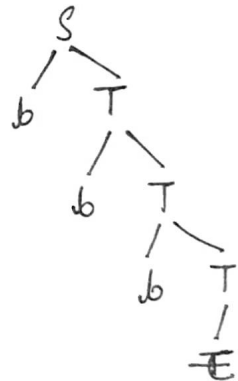


$S \rightarrow sb/b$

$S \rightarrow bT$

$T \rightarrow \epsilon/bT$

$w = bbb$



1 Start from Start Symbol

goes to string

2 Perform Left most derivation

With BackTrack

Brute force Method → (try all possibility)

→ Costly

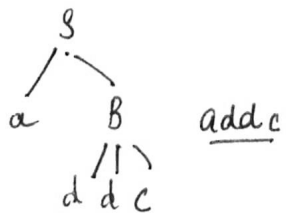
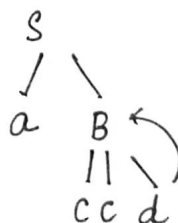
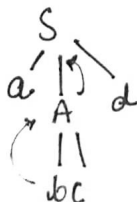
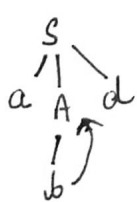
→  $O(2^n)$

$S \rightarrow aAd/aB$

$A \rightarrow b/bc$

$B \rightarrow cd/dde$

$w \rightarrow addc$

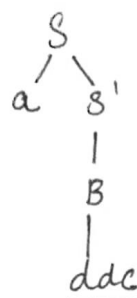
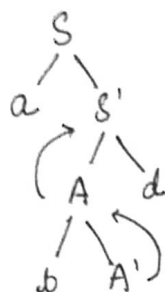


Due to Heavy cost of Brute force we developed Recursive Descent Parser.  
Brute force method is programmed using recursion.

### General RDP (Using Backtrack)

- Slightly Smarter than Brute force
- Use Backtracking

Eg →  $S \rightarrow a s'$   
 $s' \rightarrow A d / B$   
 $A \rightarrow b A'$   
 $A' \rightarrow c / E$   
 $B \rightarrow c d / d d c$



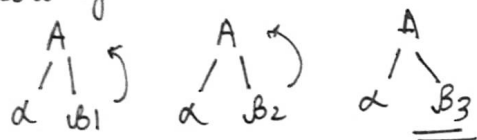
it is smarter. so it will check the next input with the string and remove the unused

### Left factor Grammar

$A \rightarrow \alpha \beta_1 / \alpha \beta_2 / \alpha \beta_3$

$w = \alpha \beta_3$

Now if we use Brute force we need to Backtrack again and again

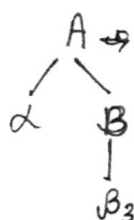


we use left factored

$A \rightarrow \alpha B$

$B \rightarrow \beta_1 / \beta_2 / \beta_3$

Now Here it become simple for parser and it won't require Backtrack



Now Eg 1 →  $S \rightarrow A a / b$

$A \rightarrow b c A / c$

$S \rightarrow b c A a / c a / b$

$A \rightarrow b c A / c$

$S \rightarrow b D / c a$

$A \rightarrow b c A / c$

$D \rightarrow c A a / e$

because Here it's nothing

Eg 2  $A \rightarrow a A b / a A d / f$

↓  
left factored

$A \rightarrow a A B / f$

$B \rightarrow b / d$

Eg-3  $S \rightarrow a / a b$

$S \rightarrow a A$

$A \rightarrow b / e$

Eg  $\rightarrow A \rightarrow (A) A / (A) Ab / (A) Aba / \epsilon / f$

$\downarrow$   
 $A \rightarrow (A) T / \epsilon / f$

$T \rightarrow A / Ab / Aba$   $T \rightarrow \epsilon / b / ba$

$\downarrow$   $\downarrow$   
 $A \rightarrow (A) T / \epsilon / f$   $A \rightarrow (A) AT / \epsilon / f$   
 $T \rightarrow$   $T \rightarrow \epsilon / bP$   
 $P \rightarrow \epsilon / a$

Eg  $\rightarrow S \rightarrow jets / ietses / a / b$

$S \rightarrow ietsP / a / b$

$P \rightarrow \epsilon / es$

Eg  $\rightarrow S \rightarrow Sa / b / bc$

$S \rightarrow aPbT / bTbT$

$T \rightarrow \epsilon / aT$

$\downarrow$

$S \rightarrow bB$

$B \rightarrow T / CT \Rightarrow B \rightarrow \epsilon / aT / CT$

$\downarrow$   
 $S \rightarrow bB$   
 $B \rightarrow \epsilon / aT / CT$

Eg  $\rightarrow$  Lecture 7/18

Without Backtrack

Note  $\rightarrow$  If grammar is Ambiguous CFG then it can be Non-left factored

Eg -  $S \rightarrow A/a$   $\} S \rightarrow a/a$   
 $A \rightarrow a$  common prefix

$S$   
 $\downarrow$   
 $a$   
 $\hline$   
 $S$   
 $\downarrow$   
 $A$   
 $\downarrow$   
 $a$   
 $\hline$

If grammar is left recursive  $\Rightarrow$  Nonleft factored

Common Prefix = Nonleft factored

Recursive Descent Parser  $\rightarrow$

(Pugnam) without Backtrack

$E \rightarrow idt$

$f \rightarrow *id f / \epsilon$

$w = id * id * id$



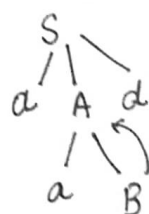
Backtrack depend on grammar.

$S \rightarrow aAd / ab$

$A \rightarrow a / ab$

$B \rightarrow ccd / ddc$

$w = addc$



We are Backtracking due to Non-left factored grammar

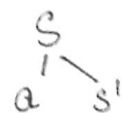
$S \rightarrow aS'$

$S' \rightarrow Ad / B$

$A \rightarrow aP$

$P \rightarrow \epsilon / B$

$B \rightarrow ccd / ddc$



Even though grammar is Left factored it can be backtracking

$\therefore$  We required LL1 parser

$\rightarrow$  Unambiguous

$\rightarrow$  Nonleft Recursive

$\rightarrow$  Left factored

## LL1 (Pause)

Input string

a b a b \$

read only tape

head only Head

LL1 Parsing Algorithm

I

LL1 Parsing Table

II

Leftmost Derivation

LL(1)

Scan Left to Right

look at  $\sigma$  ahead Symbol

(One input symbol at a time)

LL1 Parsing Table

Explicit Stack

Construct LL(1) P.T using  
1) first()  
2) follow()

first ( $\alpha$ )  $\rightarrow$  Set of ~~all~~ ~~non~~ terminals

1)  $A \rightarrow a|b$

$\text{first}(A) = \{a, b\}$

2)  $A \rightarrow a|b| \epsilon$

$\text{first}(A) = \{a, b, \epsilon\}$

3)  $S \rightarrow aA| \epsilon$   $A \rightarrow b$

$\text{first}(S) = \{a, \epsilon\}$

$\text{first}(A) = \{b\}$

4)  $S \rightarrow Aa$

$A \rightarrow bc| \epsilon$

$\text{first}(S) = \{a, b\}$

$\text{first}(A) = \{b, \epsilon\}$

FIRST And follow functions dec -8

$\text{FIRST}(\alpha) : \rightarrow$  first of  $\alpha$  gives set of all terminals that begin in string derived from  $\alpha$

$\alpha \rightarrow abc/bcd/ade$   $\text{first}(\alpha) = \{a, b, c\}$

RULES

1.  $\text{FIRST}(\alpha) = \alpha$  if  $\alpha$  is terminal

2.  $\text{FIRST}(\epsilon) = \epsilon$

3.  $\text{FIRST}(\alpha)$

$\alpha \rightarrow X_1 X_2 X_3$

$\text{FIRST}(\alpha) \rightarrow \text{FIRST}(X_1 X_2 X_3)$

$\text{FIRST}(X_1)$  if  $X_1 \rightarrow \epsilon$

then  $\text{FIRST}(X_2)$  if  $X_2 \rightarrow \epsilon$

then  $\text{FIRST}(X_3)$  if

Que 1  $\epsilon \rightarrow TE'$   
 $\epsilon' \rightarrow \epsilon / + TE'$   
 $T \rightarrow f T'$   
 $T' \rightarrow \epsilon / * f T'$   
 $f \rightarrow id / ( \epsilon )$

$FIRST(\epsilon) = FIRST(T) \rightarrow FIRST(f) \rightarrow \text{del } FIRST(id) \cup FIRST(( ) = \{id, ( \}$   
 $FIRST(\epsilon') \rightarrow FIRST(\epsilon) \cup FIRST(+TE') = \{ \epsilon, + \}$

$FIRST(T) = \{id, ( \}$   
 $FIRST(T') = \{ \epsilon, * \}$   
 $FIRST(f) = \{id, ( \}$

Que 7  $S \rightarrow AaAb / BbBa$   
 $A \rightarrow \epsilon$   
 $B \rightarrow \epsilon$

Que 2  $S \rightarrow aA / bB$   
 $A \rightarrow AB / C$   
 $B \rightarrow cB / a$   
 $A \rightarrow \epsilon T$   
 $T \rightarrow \epsilon / BT$

|   | $FIRST( )$   |
|---|--------------|
| S | a, b         |
| A | <del>c</del> |
| B | c, a         |

|   | $FIRST( )$ |
|---|------------|
| S | a b        |
| A | $\epsilon$ |
| B | $\epsilon$ |

Que 3  $S \rightarrow AB$   
 $A \rightarrow aA / \epsilon$   
 $B \rightarrow bB / \epsilon$

| $FIRST$          |   |
|------------------|---|
| a, b, $\epsilon$ | S |
| a, $\epsilon$    | A |
| b, $\epsilon$    | B |

Que 8  $S \rightarrow AaB / \epsilon$   
 $A \rightarrow BbA / a$   
 $B \rightarrow cb / d$   
 $C \rightarrow e / f$

Que 4  $S \rightarrow AaB$   
 $A \rightarrow cA / \epsilon$   
 $B \rightarrow dB / e$

|   | $FIRST( )$    |
|---|---------------|
| S | c, a          |
| A | c, $\epsilon$ |
| B | d, e          |

|   | $FIRST( )$         |
|---|--------------------|
| S | $\epsilon$ e f a d |
| A | a d e f            |
| B | d e f              |
| C | e f                |

Que 5  $S \rightarrow AaB / Ba$   
 $A \rightarrow Ba / c$   
 $B \rightarrow aB / \epsilon$

|   | $FIRST( )$          |
|---|---------------------|
| S | c, a, <del>a</del>  |
| A | a, <del>a</del> , c |
| B | a, $\epsilon$       |

Que 9  $S \rightarrow AB$   
 $A \rightarrow aA / b / \epsilon$   
 $B \rightarrow c / d / \epsilon$

|   | $FIRST( )$         |
|---|--------------------|
| S | a b c d $\epsilon$ |
| A | a b $\epsilon$     |
| B | c d $\epsilon$     |

Que 6  $S \rightarrow ABCDE$   
 $A \rightarrow a / \epsilon$   
 $B \rightarrow b / \epsilon$   
 $C \rightarrow c / \epsilon$   
 $D \rightarrow d / \epsilon$   
 $E \rightarrow \epsilon$

|   | $FIRST( )$         |
|---|--------------------|
| S | a b c d $\epsilon$ |
| A | a $\epsilon$       |
| B | b $\epsilon$       |
| C | c $\epsilon$       |
| D | d $\epsilon$       |
| E | $\epsilon$         |

Que 10  $S \rightarrow (L) / a$   
 $L \rightarrow SL'$   
 $L' \rightarrow \epsilon / , SL'$

|    | $FIRST( )$     |
|----|----------------|
| S  | ( a            |
| L  | ( a            |
| L' | $\epsilon$ , , |

due 11

$$S \rightarrow aBDh$$

$$B \rightarrow c\epsilon$$

$$C \rightarrow b\epsilon$$

$$D \rightarrow Ef$$

$$E \rightarrow g/\epsilon$$

$$f \rightarrow f/\epsilon$$

|   | first()        |
|---|----------------|
| S | a              |
| B | c              |
| C | b $\epsilon$   |
| D | g f $\epsilon$ |
| E | g $\epsilon$   |
| f | f $\epsilon$   |

due 1

$$S \rightarrow a/b/\epsilon$$

follow(S)  $\Rightarrow$  \$

first(S)  $\Rightarrow$  {a, b,  $\epsilon$ }

due 2

$$S \rightarrow aA/\epsilon$$

$$A \rightarrow b$$

|   | follow() |
|---|----------|
| S | \$       |
| A | \$       |

due 3

$$S \rightarrow aAB/\epsilon$$

$$B \rightarrow \epsilon/\epsilon$$

$$A \rightarrow b/\epsilon$$

|   | follow()        |
|---|-----------------|
| S | \$              |
| A | c $\epsilon$ \$ |
| B | \$              |

due 4

$$S \rightarrow AB$$

$$A \rightarrow aA/\epsilon$$

$$B \rightarrow bB/\epsilon$$

|   | follow() |
|---|----------|
| S | \$       |
| A | b \$     |
| B | \$       |

due 12

$$S \rightarrow ACB / CbB / Ba$$

$$A \rightarrow da / BC$$

$$B \rightarrow g/\epsilon$$

$$C \rightarrow h/\epsilon$$

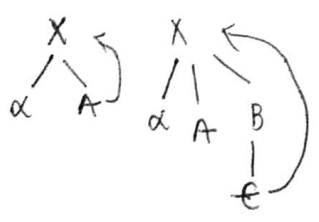
|   | first()                    |
|---|----------------------------|
| S | d, g, h, $\epsilon$ , b, a |
| A | d, g, h, $\epsilon$        |
| B | g, $\epsilon$              |
| C | h, $\epsilon$              |

FOLLOW functions lect - 9

follow(A)  $\rightarrow$  Set of all terminal that may follow immediately to right of A.

RULE

- if A is start symbol of grammar then follow(A) is minimum \$
- if  $X \rightarrow \alpha AB$  follow(A) = first(B)
- $X \rightarrow \alpha A / \alpha AB$   
 $B \rightarrow \epsilon$   
follow(A) = follow(X)



due 5

$$E \rightarrow TE'$$

$$E' \rightarrow \epsilon / +TE'$$

$$T \rightarrow fT'$$

$$T' \rightarrow \epsilon / *fT'$$

$$f \rightarrow id / (\epsilon)$$

|    | follow()             | first()      |
|----|----------------------|--------------|
| E  | $\epsilon$ ) \$      | id (         |
| E' | $\epsilon$ ) \$      | $\epsilon$ + |
| T  | + <del>id</del> ) \$ | id (         |
| T' | + <del>id</del> ) \$ | $\epsilon$ * |
| f  | \$ * ) +             | id (         |

Que 6  $S \rightarrow (L) / a$

$L \rightarrow SL'$

$L' \rightarrow \epsilon / , SL'$

|    | first()      | follow() |
|----|--------------|----------|
| S  | ( a          | \$ , )   |
| L  | ( a          | )        |
| L' | $\epsilon$ , | )        |

Check for Both

Que 7  $S \rightarrow AaAb / BbBa$

$A \rightarrow \epsilon$

$B \rightarrow \epsilon$

|   | first()    | follow() |
|---|------------|----------|
| S | a, b       | \$       |
| A | $\epsilon$ | a b      |
| B | $\epsilon$ | b a      |

Que 8

$S \rightarrow Aa$

$A \rightarrow bcd / \epsilon$

|   | first()       | follow() |
|---|---------------|----------|
| S | b, a          | \$       |
| A | b, $\epsilon$ | a        |

Que 9

$S \rightarrow ABCDE$

$A \rightarrow a / \epsilon$

$B \rightarrow b / \epsilon$

$C \rightarrow c / \epsilon$

$D \rightarrow d / \epsilon$

$E \rightarrow \epsilon$

|   | first           | follow   |
|---|-----------------|----------|
| S | abcd $\epsilon$ | \$       |
| A | a $\epsilon$    | b d \$ c |
| B | b $\epsilon$    | c \$ d   |
| C | c $\epsilon$    | d \$     |
| D | d $\epsilon$    | \$       |
| E | $\epsilon$      | \$       |

Que 10

$S \rightarrow AaB / Ba$

$A \rightarrow B / b$

$B \rightarrow aB / \epsilon$

|   | first()          | follow() |
|---|------------------|----------|
| S | b, a, $\epsilon$ | \$       |
| A | a, b             | a        |
| B | a, $\epsilon$    | \$, a, c |

Que 11  $S \rightarrow ACB / cbB / Ba$

$A \rightarrow da / BC$

$B \rightarrow g / \epsilon$

$C \rightarrow h / \epsilon$

|   | first()              | follow() |
|---|----------------------|----------|
| S | d g h $\epsilon$ b a | \$       |
| A | d g h $\epsilon$     | \$ h g   |
| B | g $\epsilon$         | \$ a h g |
| C | h $\epsilon$         | \$ g b h |

Que 12

$S \rightarrow aBDh$

$B \rightarrow cC$

$C \rightarrow bc / \epsilon$

$D \rightarrow \epsilon f$

$E \rightarrow g / \epsilon$

$f \rightarrow f / \epsilon$

|   | first()        | follow() |
|---|----------------|----------|
| S | a              | \$       |
| B | c              | g f h    |
| C | b $\epsilon$   | g f h    |
| D | g f $\epsilon$ | h        |
| E | g $\epsilon$   | h f      |
| f | f $\epsilon$   | h        |

Que 13

$S \rightarrow AaAb / BbBa$

$A \rightarrow c / \epsilon$

$B \rightarrow d / \epsilon$

|   | first()      | follow() |
|---|--------------|----------|
| S | a c d b      | \$       |
| A | c $\epsilon$ | a b      |
| B | d $\epsilon$ | b a      |

Que 14

$S \rightarrow AaB / \epsilon$

$A \rightarrow BbA / a$

$B \rightarrow Db / d$

$D \rightarrow e / f$

|   | first()                 | follow() |
|---|-------------------------|----------|
| S | $\epsilon$ , a, d, e, f | \$       |
| A | a, d, e, f              | a        |
| B | d, e, f                 | \$, b    |
| D | e, f                    | b        |

# How to Construct LL(1) Parsing Table. Construction algorithm :-

for each production  $A \rightarrow \alpha$  repeat following two steps

1. Add  $A \rightarrow \alpha$  under  $M[A, b]$  where  $b \in \text{first}(\alpha)$
2. if  $\text{first}(\alpha)$  contains  $\epsilon$  then, add  $M[A, c]$  where  $c \in \text{follow}(A)$

Ques Construct LL(1) Parsing Table for following grammar

$$\begin{aligned} E &\rightarrow E + T / T & E &\rightarrow TE' \\ T &\rightarrow T * f / f & E' &\rightarrow \epsilon / + TE' \\ f &\rightarrow \text{id} & T &\rightarrow f T' \\ & & T' &\rightarrow \epsilon / * f T' \\ & & f &\rightarrow \text{id} / (E) \end{aligned}$$

|              |    | Terminals                 |                         |                           |                           |    |                           |
|--------------|----|---------------------------|-------------------------|---------------------------|---------------------------|----|---------------------------|
|              |    | +                         | *                       | (                         | )                         | id | \$                        |
| Variable     | M  |                           |                         | $E \rightarrow TE'$       | $E \rightarrow TE'$       |    |                           |
|              | E  |                           |                         |                           |                           |    | $E' \rightarrow \epsilon$ |
|              | E' | $E' \rightarrow + TE'$    |                         |                           | $E' \rightarrow \epsilon$ |    |                           |
|              | T  |                           |                         | $T \rightarrow f T'$      | $T \rightarrow f T'$      |    |                           |
|              | T' | $T' \rightarrow \epsilon$ | $T' \rightarrow * f T'$ | $T' \rightarrow \epsilon$ |                           |    | $T' \rightarrow \epsilon$ |
| Non Terminal | f  |                           |                         | $f \rightarrow (E)$       | $f \rightarrow \text{id}$ |    |                           |
|              |    |                           |                         |                           |                           |    |                           |

①  $E \rightarrow TE' \quad M[A, b]$

$A = E$   
 $b = \text{first}(TE')$   
 $= \text{id} ($

②  $E' \rightarrow \epsilon \quad M[A, c]$

$c \in \text{follow}(E')$

$E' \rightarrow + TE'$

$M[A, b]$

$A = E'$

$b = +$

③  $T \rightarrow f T'$

$M[A, b]$

$A = T$

$b = \text{id} ($

④  $T' \rightarrow \epsilon$

$M[A, c]$

$A = T'$

$c = \text{follow}(T')$

$= + \$$

⑤  $T' \rightarrow * f T'$

$M[A, b]$

⑥  $f \rightarrow \text{id}$

$f \rightarrow (E)$

S.

Ans

Since each entries contain single production then the given grammar is LL1

Ques 2 Construct LL1 Parsing Table for following grammar

$S \rightarrow (L) a$

$L \rightarrow S L'$

$L' \rightarrow \epsilon / , S L'$

$$S \rightarrow (L)/a$$

$$L \rightarrow SL'$$

$$L' \rightarrow \epsilon / , SL'$$

| M  |                     | (                         | ,                      | \$ | a                   |
|----|---------------------|---------------------------|------------------------|----|---------------------|
| S  | $S \rightarrow (L)$ | —                         | —                      | —  | $S \rightarrow a$   |
| L  | $L \rightarrow SL'$ | <del>—</del>              | —                      | —  | $L \rightarrow SL'$ |
| L' | —                   | $L' \rightarrow \epsilon$ | $L' \rightarrow , SL'$ | —  | —                   |

①  $S \rightarrow (L)$   
 $M[A, b]$

②  $S \rightarrow a$   
 $M[A, b] = M[S, a]$

$A = S$   
 $b = \text{first}((L))$   
 $= [$

③  $L \rightarrow SL'$   
 $M[A, b]$   
 $M[L, \text{first}(SL')] \rightarrow \emptyset [L, \{ ( a \}]$

④  $L' \rightarrow \epsilon$   
 $M[A, c]$   
 $M[A, \text{follow}(L')] = [A, \{ ) \}]$

Since each entry contain single production so grammar is LL1.

⑤  $L' \rightarrow \epsilon, SL'$   
 $M[A, b] = [L', ,]$

If any entry contain multiple production then it is not LL1

Que 3 Construct LL1 P.T

$$S \rightarrow AaAb / BbBa$$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

$$S \rightarrow AaAb$$

$$M[A, b] = [S, \{a\}]$$

|   | a                        | b                        | \$ |
|---|--------------------------|--------------------------|----|
| S | $S \rightarrow AaAb$     | $S \rightarrow BbBa$     | —  |
| A | $A \rightarrow \epsilon$ | $A \rightarrow \epsilon$ | —  |
| B | $B \rightarrow \epsilon$ | $B \rightarrow \epsilon$ | —  |

This is also LL1.

Que 4  $S \rightarrow asa / \epsilon$

$$S \rightarrow asa$$

$$M[A, b] = [S, \{a\}]$$

$$S \rightarrow \epsilon$$

$$M[A, c] = [S, a]$$

|   | a                                               | \$                       |
|---|-------------------------------------------------|--------------------------|
| S | $S \rightarrow asa$<br>$S \rightarrow \epsilon$ | $S \rightarrow \epsilon$ |

Since each entry doesn't contain single production.  $\therefore$  Grammar isn't LL1

Que 5

$$S \rightarrow A/a$$

$$A \rightarrow a$$

$$S \rightarrow A$$

$$M[A, b] = [S, \{a\}]$$

Ambiguous also Not Left factored

|   | a                   | \$ |
|---|---------------------|----|
| S | $S \rightarrow A/a$ | —  |
| A | $A \rightarrow a$   | —  |

Not LL1 Grammar

Que 6  $S \rightarrow aA$   
 $A \rightarrow b/\epsilon$

- ①  $S \rightarrow aA$
- ②  $A \rightarrow b$
- ③  $A \rightarrow \epsilon$

| M | a                  | b                 | \$                       |
|---|--------------------|-------------------|--------------------------|
| S | $S \rightarrow aA$ |                   |                          |
| A |                    | $A \rightarrow b$ | $A \rightarrow \epsilon$ |

Since each entry is singular therefore Grammar is LL1

Que 7  $S \rightarrow AB$   
 $A \rightarrow a/\epsilon$   
 $B \rightarrow b/\epsilon$

- ①  $S \rightarrow AB$

$[S, \{a, b, \epsilon\}]$

Become follow of S

|   | a                  | b                        | \$                       |                                                |
|---|--------------------|--------------------------|--------------------------|------------------------------------------------|
| S | $S \rightarrow AB$ | $S \rightarrow AB$       | $S \rightarrow AB$       | <del><math>S \rightarrow AB</math></del>       |
| A | $A \rightarrow a$  | $A \rightarrow \epsilon$ | $A \rightarrow \epsilon$ | <del><math>A \rightarrow \epsilon</math></del> |
| B | -                  | $B \rightarrow b$        | $B \rightarrow \epsilon$ | $\epsilon$                                     |

- ②  $A \rightarrow a$
- ③  $A \rightarrow \epsilon$
- ④  $B \rightarrow b$
- ⑤  $B \rightarrow \epsilon$

lecture 11 / 18

P.T is storing Grammar in form of Table.

Now we need to check input string is syntactically correct or not using stack and LL1 Parsing algorithm.

SHORT TRICK FOR CHECKING WHETHER GRAMMAR IS LL1 OR NOT

- LL(1) .
1. Unambiguous
  2. non left Recursive / Right Recursive
  3. Left factored

①  $A \rightarrow \alpha_1/\alpha_2$  if  $FIRST(\alpha_1) \cap FIRST(\alpha_2) = \phi$  ..  
 then grammar is LL1

②  $A \rightarrow \alpha_1/\epsilon$  if  $FIRST(\alpha_1) \cap follow(A) = \phi$   
 then grammar is LL1.

Eg  $\rightarrow S \rightarrow ab/cd$

$first(ab) \cap first(cd) = a \cap c = \phi$  LL1.

Eg2  $\rightarrow S \rightarrow bSb/\epsilon$   $FIRST(bSb) \cap follow(S)$

$= b \cap b = b \neq \phi$  Not LL1.

Eg3  $\rightarrow S \rightarrow (L)/a$   
 $L \rightarrow SL'$

$L' \rightarrow \epsilon/, SL'$

$first((L)) \cap first(a) = ( \cap a = \phi$

$first(, , SL') \cap follow(L') = , \cap ) = \phi$

$\therefore$  In LL1

lecture - 11 / 18

LL 1 Parser

TOP DOWN PARSER

If Grammar contains

①  $A \rightarrow \alpha_1 / \alpha_2 / \alpha_3$  Here all  $\alpha$  doesn't contain null ( $\epsilon$ )  
then

$$\left. \begin{array}{l} \text{first}(\alpha_1) \cap \text{first}(\alpha_2) \\ \text{first}(\alpha_2) \cap \text{first}(\alpha_3) \\ \text{first}(\alpha_1) \cap \text{first}(\alpha_3) \end{array} \right\} \text{Pairwise disjoint}$$

then grammar is LL1.

②  $A \rightarrow \alpha_1 / \alpha_2 / \epsilon$

$$\text{first}(\alpha_1) \cap \text{first}(\alpha_2) = \phi$$

$$\text{first}(\alpha_1) \cap \text{follow}(A) = \phi \quad \text{then grammar is LL1.}$$

$$\text{first}(\alpha_2) \cap \text{follow}(A) = \phi$$

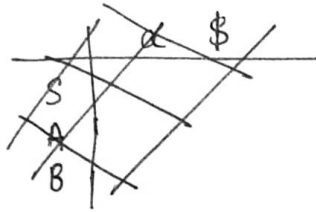
Eg  $\rightarrow S \rightarrow AB$   
 $A \rightarrow aA / a$  ] - Not LL1.

2)  $E \rightarrow \pm TE / \pm$  - yes

$$T \rightarrow (f)$$

$$f \Rightarrow id$$

it is LL1.



Note:  $\rightarrow$  Every Ambiguous grammar is not LL1.

2. Every unambiguous grammar need not to be LL1.

3. Every LL1 grammar is unambiguous.

LL(1) Algorithm  $\rightarrow$  1. Let  $X$  is top of stack and  $a$  is input symbol or look ahead symbol.

if ( $X = a = \$$ ) successful completion of parsing (accept)

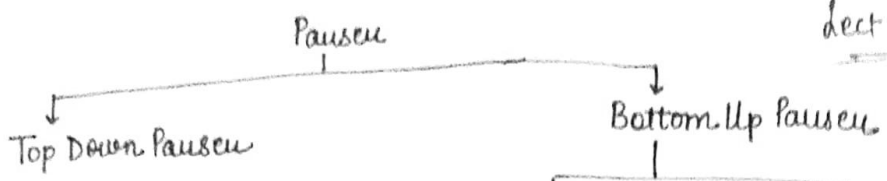
if ( $X = a \neq \$$ ) then pop Top of Stack and Increment Input Pointer.

if ( $X$  is a variable / nonterminal) then see p.T.M. [ $X, a$ ]

$X \rightarrow xyz$  then pop  $X$  and push  $xyz$  in reverse order.

if  $M[X, a] = \text{blank}$  then Error.

BOTTOM UP PARSER



Bottom Up Parser :-

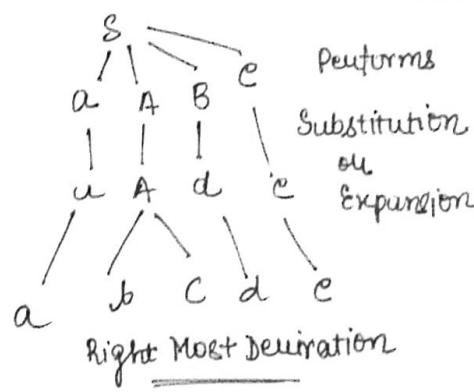
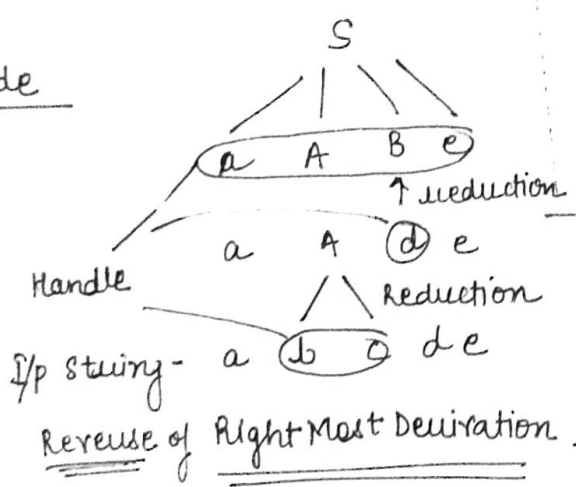
How BUP Work?

$S \rightarrow aABe$   
 $A \rightarrow bc$   
 $B \rightarrow d$   
 $w = \underline{abcde}$

It can Pass both Ambiguous & Non Ambiguous Grammar

LR  $\rightarrow$  Reverse of Right Most Derivation  
 Scan Left to Right

SLR - Simple LR  
 LALR - Look Ahead LR  
 CLR - Canonical LR



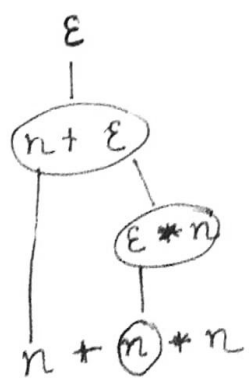
Handle - Part of i/p string that matches Right Hand Side of production

Handle Replacing - Replacing the variable with its LHS variable is known as Handle Replacing

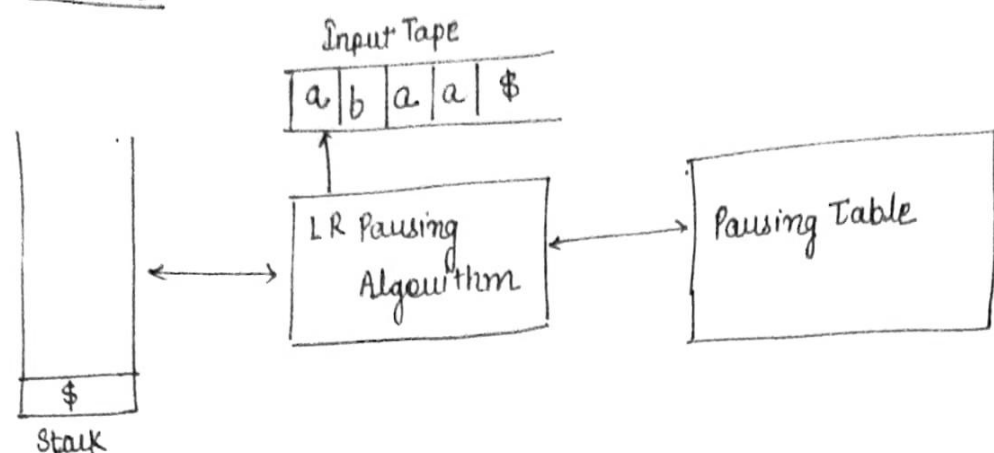
Eg  $\rightarrow$  Consider a grammar

$E \rightarrow E + n / E * n / n$   
 $w = n + n * n$

no of handles are - 3



## LR Parser



★ ★  $\forall$  Parsing Algorithms is same i.e. Reverse of Right Most Derivation.  
only parsing Table will be different for all algorithms.

LR(0) SLR(1)

CLR(1)

LALR(1)

CLR(1) is the Most powerful but Mostly used is LALR(1)

in terms of accepting more no of language.

Powerful Parsing  $\boxed{LR(0) < SLR(1) < LALR(1) < CLR(1)}$

LR Parsing can parse only unambiguous Grammar.

$\boxed{\text{Top Down} < \text{Bottom Up}}$  due to accepting more language.

LR(0) Parsing  $\xrightarrow{\text{look ahead symbol}}$

$\downarrow$  Reverse of RMD

Scan left to Right

Parsing Table of LR(0) Construction  $\rightarrow$

1. Construct Augmented Grammar
2. Construct Canonical set of items using closure and goto function.
3. Construct DFA
4. Construct the P.T (Minimize Minimize DFA to P.T)

P.T.O  $\rightarrow$

Construct LR(0) P.T for following grammar.

$$A \rightarrow aA$$

$$A \rightarrow b$$

Step 1: Construct Augmented Grammar

$$\left. \begin{array}{l} A' \rightarrow A \\ A \rightarrow aA \\ A \rightarrow b \end{array} \right\}$$

Step 2: Make LR(0) item

$$A' \rightarrow \cdot A$$

LR(0) item

Step 3: Using closure and goto construct DFA

$$\text{closure}(A' \rightarrow \cdot A) = A' \rightarrow \cdot A$$

if after  $\cdot$  we have variable then add prod from grammar

$$A' \rightarrow \cdot aA$$

Eg-2 closure  $(A \rightarrow \cdot a b) = A \rightarrow \cdot a b$

Eg-3 closure  $(A \rightarrow A \cdot a) = A \rightarrow A \cdot a$

Eg-4 - closure  $(A \rightarrow A \cdot A) = A \rightarrow A \cdot aA$

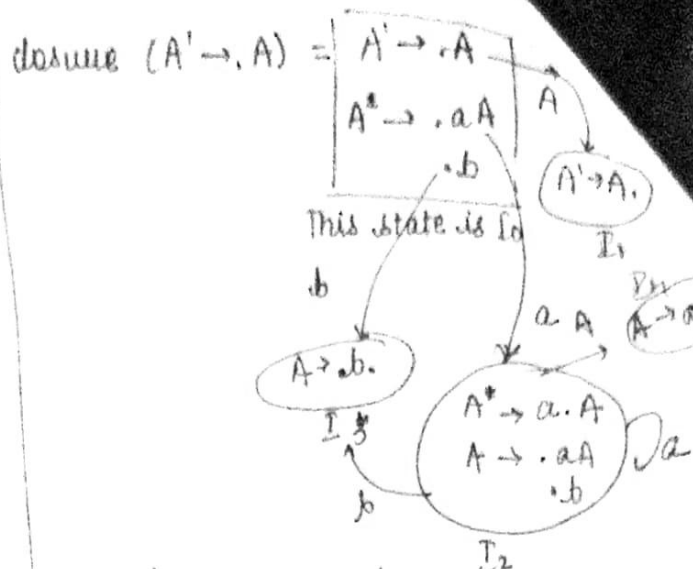
Eg-5 - closure  $(A \rightarrow AA \cdot) = A \rightarrow AA \cdot$

Goto  $\rightarrow$   $\text{Goto}(A \rightarrow x \cdot x \beta \cdot x) =$

$$A \rightarrow x \cdot x \beta$$

Eg Goto  $(A \rightarrow a \cdot A \cdot A) = A \rightarrow a A \cdot$

$$\begin{aligned} \text{Goto} \cdot (A \rightarrow \cdot aA \cdot a) &= A \rightarrow a \cdot A \\ &= A \rightarrow a \cdot aA \\ &\rightarrow a \cdot b \end{aligned}$$



|   | action |    |     | Goto part |
|---|--------|----|-----|-----------|
|   | a      | b  | \$  |           |
| 0 | s2     | s3 |     | 1         |
| 1 |        |    | acc |           |
| 2 | s2     | s3 |     | 4         |
| 3 | r2     | r2 | r2  |           |
| 4 | r1     | r1 | r1  |           |

Pausing Table

Conflict  $\rightarrow$  SR (Shift Reduce)

RR (Reduce Reduce)

LR(0) Item

Lect -14/18

Reduced item  
 $A \rightarrow b \cdot$

Shift item  
 $A \rightarrow \cdot aA$   
 $A \rightarrow \cdot a$

State item  
 $A \rightarrow a \cdot A$

SR (Shift Reduce)  $\rightarrow$

$$\begin{array}{l} A \rightarrow \cdot a \\ A \rightarrow \cdot b \end{array}$$

Both shift and reduce at same state

RR (Reduce Reduce)  $\rightarrow$

$$\begin{array}{l} A \rightarrow a \cdot \\ b \cdot \end{array}$$

\* We don't have SS Conflict because we are making DFA.

\* Goto part doesn't create any conflict

↓  
We don't have only state no entries

(\*) If any kind of conflict is there  
grammar is not LR0

Due constant LR(0) P.T for following grammar

$E \rightarrow T + E / T$  [I]  $S \rightarrow \cdot E$   
 $T \rightarrow a$   $E \rightarrow T + \cdot E / T$   
 $T \rightarrow a \cdot$

(II) Taking closure

$\text{closure}(S \rightarrow \cdot E) =$



Since there is a conflict  
∴ we get it not a LR(0)

LR(0)

1. Same DFA
2. Goto entries are same in Table
3. Shift entries are same
4. Reduction entry are different  
(Blindly put in action part)

SLR(1) (Simple LR)

1. Same DFA
2. Goto entries are same
3. Shift entries are same
4. ← →
5. Reduction entry are less as compared to LR(0)

|   | Action         |                |                | Goto |   |
|---|----------------|----------------|----------------|------|---|
|   | +              | a              | \$             | E    | T |
| 0 |                | S <sub>3</sub> |                | 1    | 2 |
| 1 |                |                | acc            |      |   |
| 2 | S <sub>4</sub> |                | r <sub>2</sub> |      |   |
| 3 | r <sub>5</sub> |                | r <sub>3</sub> |      |   |
| 4 |                | S <sub>3</sub> |                | 5    | 2 |
| 5 |                |                | r <sub>1</sub> |      |   |

Since, No conflict ∴ Grammar is SLR(1)

SLR(1) table for above problem

wherever there is a reduction  
just find follow of start symbol  
of that production like

$E \rightarrow T$  follow(E) = \$

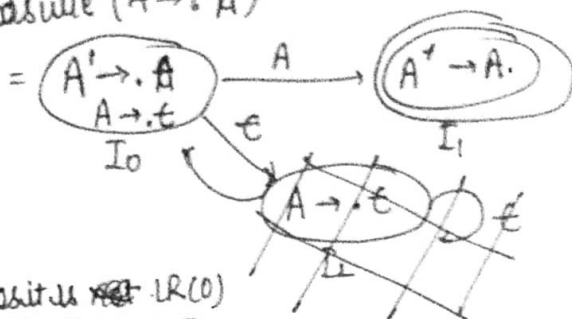
$T \rightarrow a$  follow(T) = + \$

$E \rightarrow T + E$  follow(E) = \$

Que!  $\rightarrow A \rightarrow \epsilon$  Construct LR(0) P.T

- I ~~LR(0) item~~  $A' \rightarrow \cdot A$  LR(0) item  
 II  $A' \rightarrow A$   
 III closure ( $A' \rightarrow \cdot A$ )

|   | Action     |       |  | goto |
|---|------------|-------|--|------|
|   | $\epsilon$ | \$    |  |      |
| 0 | $r_0$      | $r_0$ |  | 1    |
| 1 |            | acc   |  |      |



Not LR(0)  
 Yes it is SLR(1)

if grammar is LR(0) then it is SLR(1)

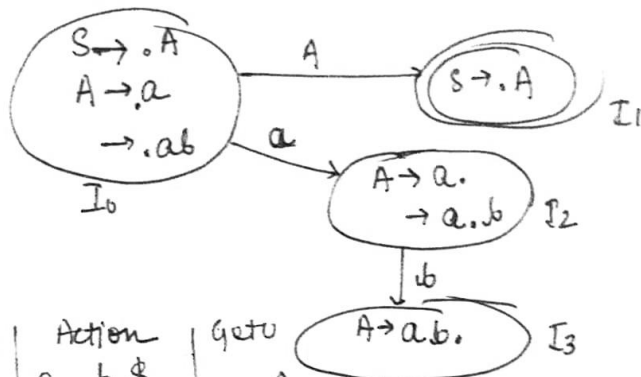
due  $A \rightarrow a/ab$  Construct LR(0) & SLR(1)

- I  $S \rightarrow A$  II  $S \rightarrow \cdot A$  III closure ( $S \rightarrow \cdot A$ ) =

$A \rightarrow a/ab$

| LR(0) | Action |           |    | goto |
|-------|--------|-----------|----|------|
|       | a      | b         | \$ |      |
| 0     | $S_2$  |           |    | 1    |
| 1     |        | acc       |    |      |
| 2     | $r_2$  | $S_3/r_4$ |    |      |
| 3     | $r_2$  | $r_2$     |    |      |

$\therefore$  it is not LR(0)



SLR(1)

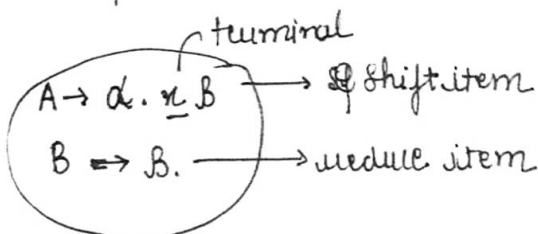
|   | Action |       |       | goto |
|---|--------|-------|-------|------|
|   | a      | b     | \$    |      |
| 0 | $S_2$  |       |       | 1    |
| 1 |        | acc   |       |      |
| 2 |        | $S_3$ | $r_4$ |      |
| 3 |        | $r_2$ |       |      |

$\therefore$  it is SLR(1)

due  $S \rightarrow SS/a$

Ambiguous

$\therefore$  No need to check for LR



- $\rightarrow$  In LR(0)  $I_i$  state we have SR conflict  
 $\rightarrow$  In SLR(1)  $\rightarrow$  first(x)  $\cap$  follow(B)  $\neq \phi$   
 then SR conflict is there.

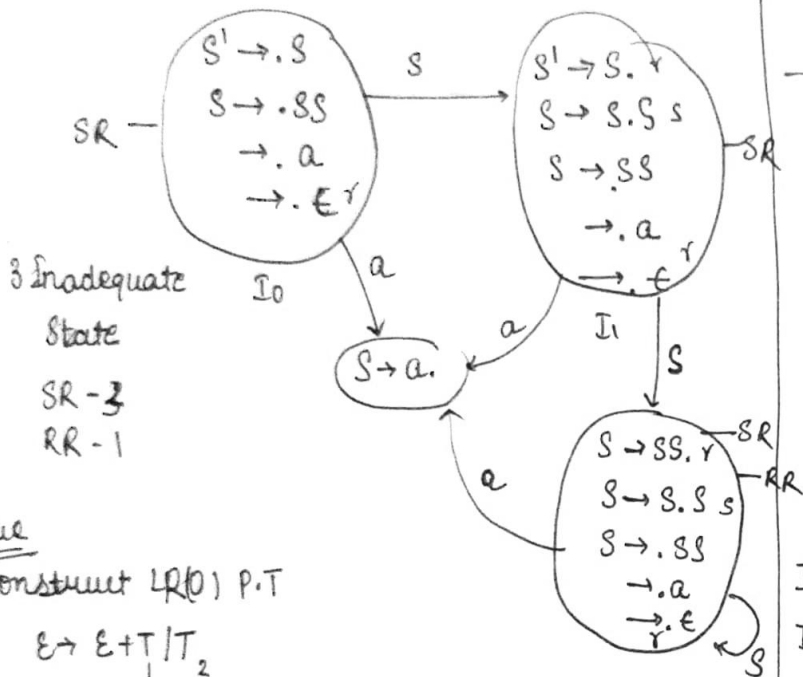
Eg  $\rightarrow$   $A \rightarrow a.$   $\rightarrow$  reduced  $\rightarrow$  In LR(0) we have RR conflict  
 $B \rightarrow B.$   $\rightarrow$  reduced  $\rightarrow$  In SLR(1) follow(A)  $\cap$  follow(B)  $\neq \phi$   
 we have RR conflict.

if a Grammar is ambiguous then  
 → Then it is not LL(1)  
 " " " " LR(0)  
 " " " " SLR(1)  
 " " " " CLR(1)  
 " " " " LALR(1)

Que  $S \rightarrow SS/a/\epsilon$  no of inadequate state and  
 How many SR and RR conflict are there?

$S \rightarrow SS/a/\epsilon$

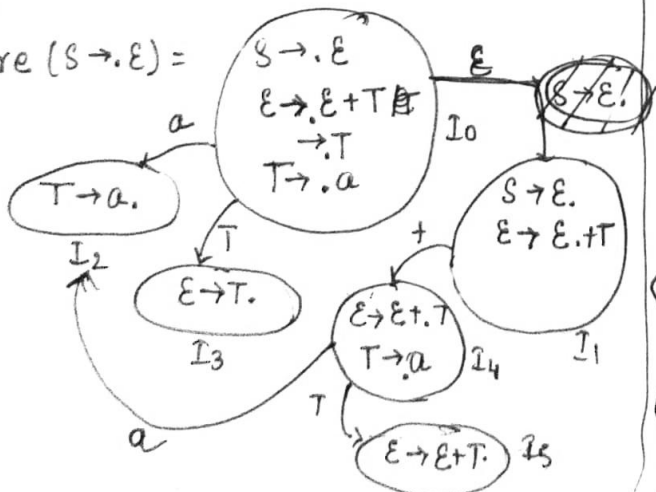
- I  $S' \rightarrow S$   
 $S \rightarrow SS/a/\epsilon$   
 II  $S' \rightarrow \cdot S$   
 III closure ( $S' \rightarrow \cdot S$ ) =



Que  
 Construct LR(0) P.T

$E \rightarrow E+T/T_2$   
 $T \rightarrow a_3$

- I  $S \rightarrow E$   
 II  $S \rightarrow \cdot E$   
 III closure ( $S \rightarrow \cdot E$ ) =



|   | a  | Action | $\phi$ | $\epsilon$ | goto | T |
|---|----|--------|--------|------------|------|---|
| 0 | S2 |        |        | 1          |      | 3 |
| 1 |    | S4     | acc    |            |      |   |
| 2 | r3 | r3     | r3     |            |      |   |
| 3 | r2 | r2     | r2     |            |      |   |
| 4 | S2 |        |        |            |      | 5 |
| 5 | r1 | r1     | r1     |            |      |   |

it is LR(0)  $\therefore$  it is also SLR(1)

Que

$S \rightarrow AaAb/BbBa$

$A \rightarrow \epsilon$

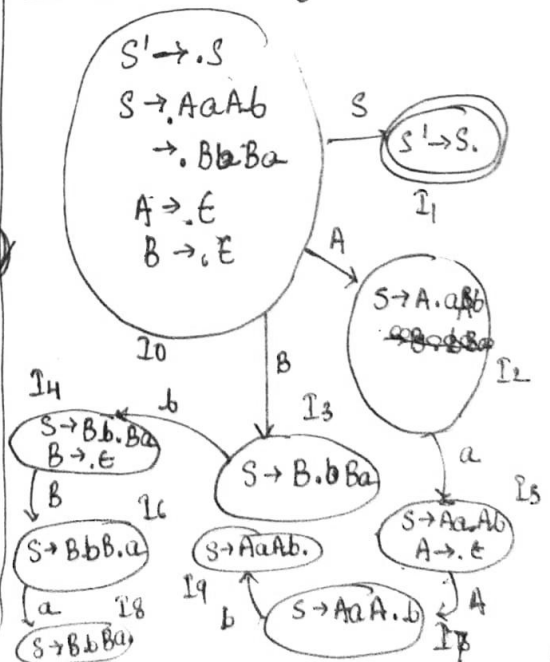
$B \rightarrow \epsilon$

|  | Action               | goto |
|--|----------------------|------|
|  | Shifted to next page |      |

- I  $S' \rightarrow S$   
 $S \rightarrow AaAb/BbBa$   
 $A \rightarrow \epsilon$   
 $B \rightarrow \epsilon$

II  $S' \rightarrow \cdot S$

III closure ( $S' \rightarrow \cdot S$ ) =



Due to RR Conflict it is not LR(0)

No of inadequate state = 1

Due Construct LR(0) & SLR(1)

$E \rightarrow E + T / T$

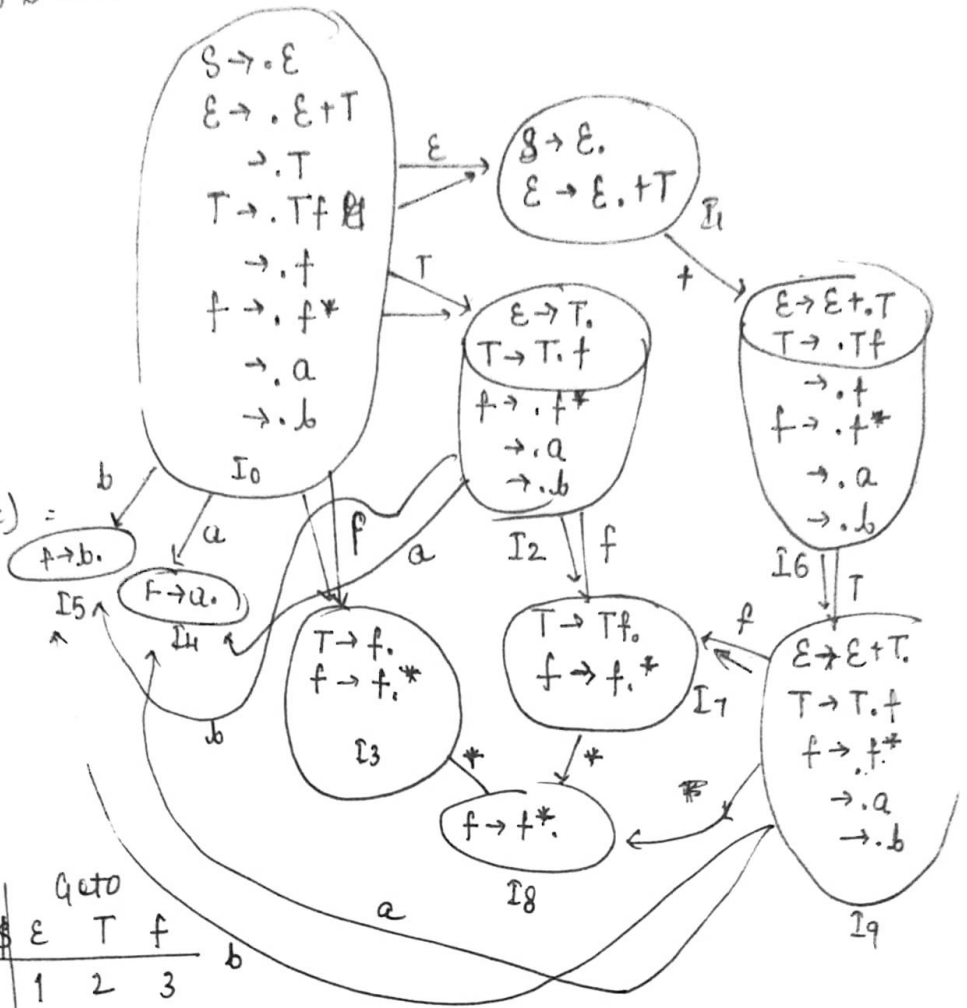
$T \rightarrow T * f / f$

$f \rightarrow f^* / a / b$

I  $S \rightarrow E$   
 $E \rightarrow E + T_1 / T_2$   
 $T \rightarrow T f_3 / f_4$   
 $f \rightarrow f^*_5 / a_6 / b_7$

II  $S \rightarrow \cdot E$

III Closure ( $S \rightarrow \cdot E$ ) =



|   | Action |       |       |   |        | Goto |   |   |
|---|--------|-------|-------|---|--------|------|---|---|
|   | a      | b     | *     | + | \$     | E    | T | f |
| 0 | S4     | S5    |       |   |        | 1    | 2 | 3 |
| 1 |        |       |       |   | S6 acc |      |   |   |
| 2 | S4/r2  | S5/r2 |       |   |        |      |   | 7 |
| 3 | r4     | r4    | S8/r4 |   |        |      |   |   |
| 4 | r6     | r6    | r6    |   |        |      |   |   |
| 5 | r7     | r7    | r7    |   |        |      |   |   |
| 6 | S4     | S5    |       |   |        |      |   |   |
| 7 | r3     | r3    | S8/r3 |   |        |      |   |   |
| 8 | r5     | r5    | r5    |   |        |      |   |   |
| 9 | S4/r1  | S5/r1 |       |   |        |      |   | 7 |

Not LR(0)

It is SLR(1)

CLR(1) / LR(1) and LALR(1)

we make LR1 item

LR(0) item, look ahead symbol

Eg  $\rightarrow S \rightarrow \underline{AA}$       \$  
LRO item      Look ahead symbol

Construct CLR(1) / LR(1) for Grammar

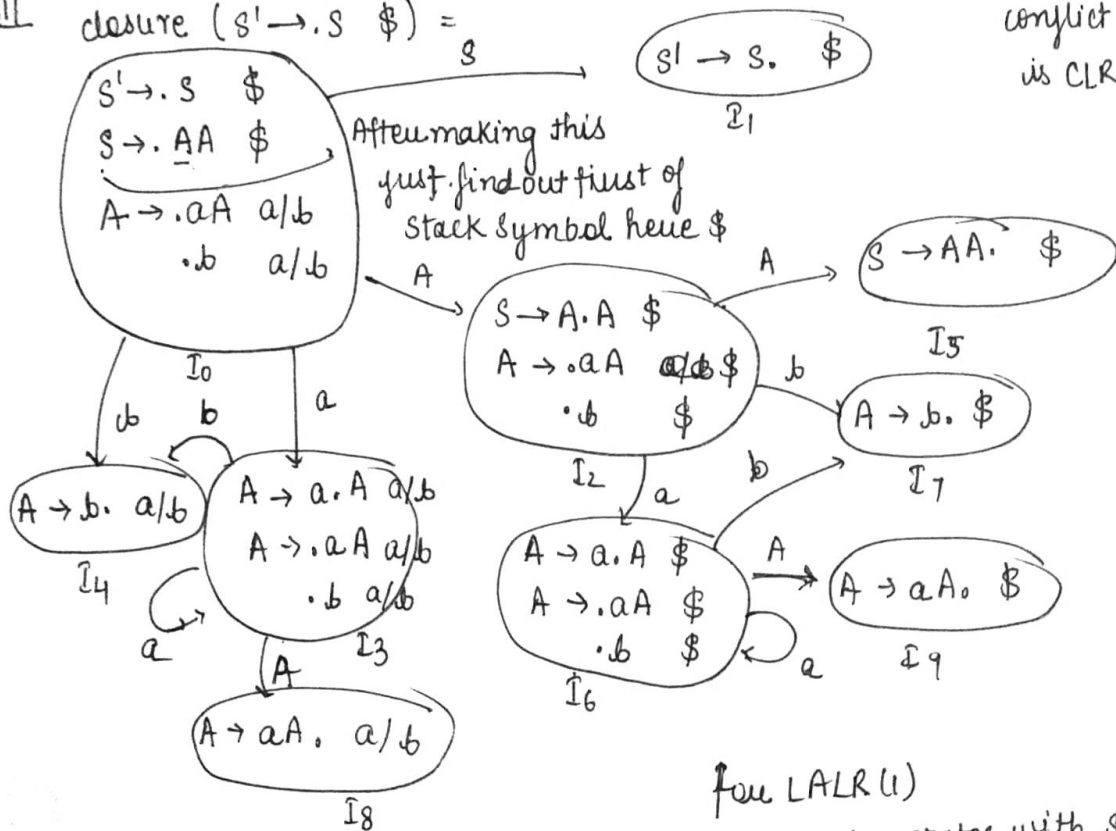
$$S \rightarrow AA$$
$$A \rightarrow aA/b$$
$$\text{I} \quad S' \rightarrow \pi S$$
$$S \rightarrow AA$$
$$A \rightarrow a_2 A / b_3$$

II make LR(1) item and construct DFA using closure and goto.

$$\left( \begin{array}{c} S' \rightarrow .S \\ \hline \text{LR(0)} \\ \text{item} \end{array} \quad \$ \right)$$

LR(1) item

III  $\text{closure}(s' \rightarrow . S \ \$) =$



Since there is no conflict this grammar is CLR(1)

For LALR (1)

we merge states with same  
production, but diff look ahead  
symbol

like I3 & I6 will merge

# LALR(1) Table for problem

|    | Action |     |     | Goto |    |
|----|--------|-----|-----|------|----|
|    | a      | b   | \$  | S    | A  |
| 0  | S36    | S47 |     | 1    | 2  |
| 1  |        |     | acc |      |    |
| 2  | S36    | S47 |     |      | 5  |
| 36 | S36    | S47 |     |      | 89 |
| 47 | r3     | r3  | r3  |      |    |
| 5  |        |     | r1  |      |    |
| 89 | r2     | r2  | r2  |      |    |

Since there is no conflict  
 $\therefore$  this is also LALR(1)

due find no of items in  $I_0$  in CLR(1) parsing Table for

$$E \rightarrow E+T/T$$

$$T \rightarrow T*f/f$$

$$f \rightarrow f*/a/b$$

$$I \rightarrow S \rightarrow E \quad E \rightarrow E+T/T \quad T \rightarrow T*f/f \quad f \rightarrow f*/a/b$$

$$II \rightarrow (S \rightarrow \cdot E \quad \$)$$

$$III \rightarrow \text{closure}(S \rightarrow \cdot E \quad \$) =$$

|                                             |                                   |                                  |                                   |
|---------------------------------------------|-----------------------------------|----------------------------------|-----------------------------------|
| $S \rightarrow \cdot E \quad \$$            | $E \rightarrow \cdot E+T \quad +$ | $E \rightarrow T \cdot \quad +$  | $T \rightarrow \cdot T*f \quad *$ |
| $E \rightarrow \cdot E+T \quad \$/+/\equiv$ | $\cdot T \quad +$                 | $\cdot f \quad *$                |                                   |
| $\cdot T \quad \$/+/\equiv$                 | $T \rightarrow \cdot T*f \quad +$ | $f \rightarrow \cdot f* \quad *$ |                                   |
| $T \rightarrow \cdot T*f \quad \$/+/*$      | $\cdot f \quad +$                 | $\cdot a \quad *$                |                                   |
| $\cdot f \quad \$/+/*$                      | $f \rightarrow \cdot f* \quad +$  | $\cdot b \quad *$                |                                   |
| $f \rightarrow \cdot f*/a/b \quad \$/+/*$   | $\cdot a \quad +$                 |                                  |                                   |
| $\cdot a \quad \$/+/*$                      | $\cdot b \quad +$                 |                                  |                                   |
| $\cdot b \quad \$/+/*$                      |                                   |                                  |                                   |
| $I_0$                                       |                                   |                                  |                                   |

due  $S \rightarrow AaAb/BbBa$

$$A \rightarrow \epsilon$$

$$B \rightarrow \epsilon$$

No Conflict is there  $\therefore$  it is CLR(1) and LALR(1)

Construct LR(0)

$S \rightarrow AA$

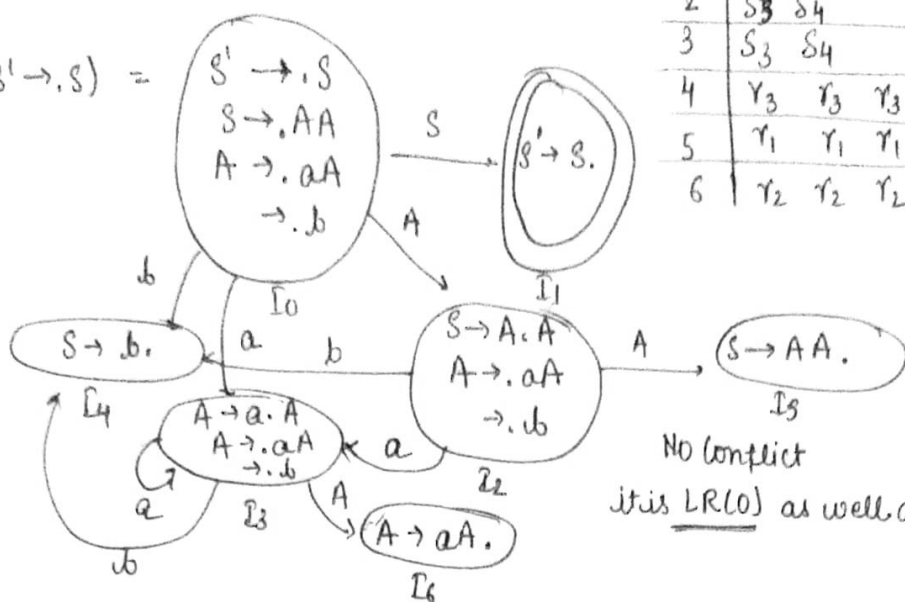
$A \rightarrow aA/b$

I  $S' \rightarrow S$

II  $S' \rightarrow \cdot S$

III Closure ( $S' \rightarrow \cdot S$ ) =

|   | a              | b              | \$             | S | A |
|---|----------------|----------------|----------------|---|---|
| 0 | S <sub>3</sub> | S <sub>4</sub> |                | 1 | 2 |
| 1 |                |                | acc            |   |   |
| 2 | S <sub>3</sub> | S <sub>4</sub> |                |   | 5 |
| 3 | S <sub>3</sub> | S <sub>4</sub> |                |   | 6 |
| 4 | r <sub>3</sub> | r <sub>3</sub> | r <sub>3</sub> |   |   |
| 5 | r <sub>1</sub> | r <sub>1</sub> | r <sub>1</sub> |   |   |
| 6 | r <sub>2</sub> | r <sub>2</sub> | r <sub>2</sub> |   |   |



No conflict  
it is LR(0) as well as SLR(1)

LR(0)      SLR(1)      CLR(1)      LALR(1)

no of state

$n_0$

$n_2$

$n_3$

$n_4$

relation  $\Rightarrow n_1 = n_2 = n_4 \leq n_3$  \*\*

Parsing table entries

- Shift Entry ( $S_i$ ) (terminals)
- State Entry ( $i$ ) (Variable)
- Reduction Entry ( $R_i$ ) ( $A \rightarrow b \cdot$ )
- Acceptance Entry (acc)
- Blank/error

Reduction Entries  $A \rightarrow B \cdot$

**LR(0)**  
action part

**SLR(1)**  
follow(A)

**CLR(1)**  
look Ahead Symbol

**LALR(1)**  
look ahead symbol

1. No of shift entries is same in LR(0), SLR(1), LALR(1) < CLR(1)

2. " " Table " " " " " " " "

3. No of state " " " " " " " "

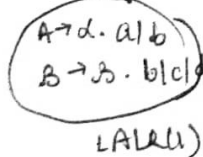
4. Reduction entries in LR(0)  $\geq$  SLR(1)  $\geq$  LALR(1)  $\geq$  CLR(1)

CLR(1)

CLR

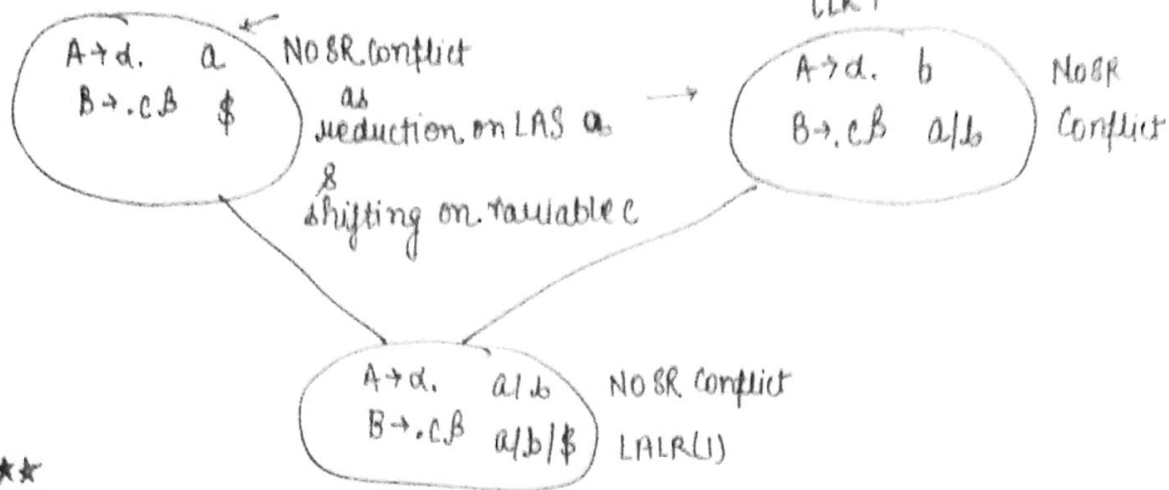


No RR conflict  
as both reductions  
occur for diff LAS.

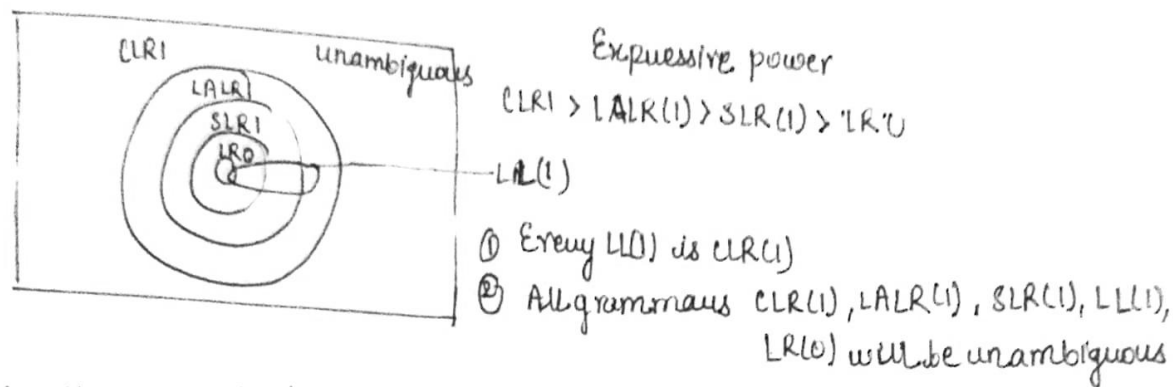


Here conflict  
as we have diff  
production reduction  
for same LAS. i.e. b

CLR(1)



★★  
If given grammar is CLR(1) but not LALR(1) then there must be RR conflict  
★★  
If LALR(1) don't have SR conflict then CLR(1) also don't have SR conflict.



Algorithm. for LR(0) :-

- if S is Top of stack and a is lookahead symbol
1. if action [S, a] = S<sub>i</sub> then shift a and i and increment input pointer
2. if action [S, a] = r<sub>j</sub> and r<sub>j</sub> is  $\alpha \rightarrow \beta$  then pop  $2 \times |\beta|$  from Top of stack and replace by  $\alpha$
3. if S<sub>m-1</sub> is the state below a then push goto [S<sub>m-1</sub>, a]
4. if action [S, a] = acc then successful completion of parsing
5. if action [S, a] = blank then error

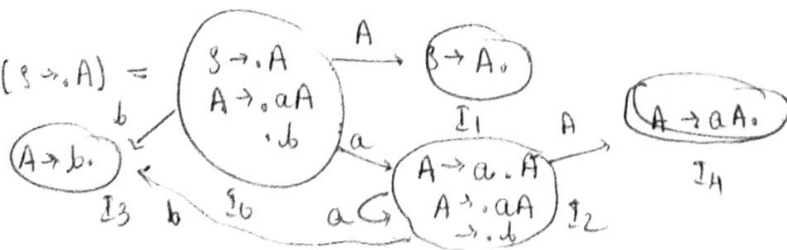
Construct LR(0) Parsing Table

$A \rightarrow aA|b$

I  $S \rightarrow A$

II  $S \rightarrow .A$

III closure ( $S \rightarrow .A$ ) =



|   | Action         |                |                | Goto |   | Input $\rightarrow$ <u>aab</u> |
|---|----------------|----------------|----------------|------|---|--------------------------------|
|   | a              | b              | \$             |      | A |                                |
| 0 | S <sub>2</sub> | S <sub>3</sub> |                |      |   |                                |
| 1 |                |                | acc            |      | 1 |                                |
| 2 | S <sub>2</sub> | S <sub>3</sub> |                |      | 4 |                                |
| 3 | r <sub>2</sub> | r <sub>2</sub> | r <sub>2</sub> |      |   |                                |
| 4 | r <sub>1</sub> | r <sub>1</sub> | r <sub>1</sub> |      |   |                                |

|    |
|----|
|    |
|    |
| 0  |
| \$ |

| input          | stack     |                                                  |
|----------------|-----------|--------------------------------------------------|
| <u>a</u> ab\$  | \$0       | 0 on a gives S <sub>2</sub> (shift a, 2)         |
| a <u>a</u> b\$ | \$0a2     | increment i/p pointer                            |
| aa <u>b</u> \$ | \$0a2a2   | Shift(a, 2) = S <sub>2</sub> , increment ip      |
| aab <u>\$</u>  | \$0a2a2b3 | Shift(b, 3) = S <sub>3</sub>                     |
|                |           | increment ip                                     |
| aab\$          | \$0a2a2A  | 3 on \$ is reduction (r <sub>2</sub> )           |
|                |           | <u>A</u> $\rightarrow$ b pop (2 x length(b))     |
| aab\$          | \$0a2a2A4 | Now if S <sub>n-1</sub> is state below & we push |
|                |           | [S <sub>n-1</sub> , d] i.e. [2, A] = 4           |
| aab\$          | \$0a2A    | Now 4 on \$ = r <sub>1</sub>                     |
|                |           | <u>A</u> $\rightarrow$ aA length = 2             |
| aab\$          | \$0a2A    | $\therefore$ pop 2 x 2 items and replace with aA |
| aab\$          | \$0a2A4   | Now, push [2, A] = 4                             |
|                |           | [4, \$] = r <sub>1</sub> pop 2 x len(aA)         |
| aab\$          | \$0A      | = 4 items and replace with A                     |
|                |           | push [0, A] = 1                                  |
| aab\$          | \$0A1     | Now [1, \$] = acc                                |
|                |           | $\therefore$ the string is accepted              |

YACC tool  $\rightarrow$  LALR(1)

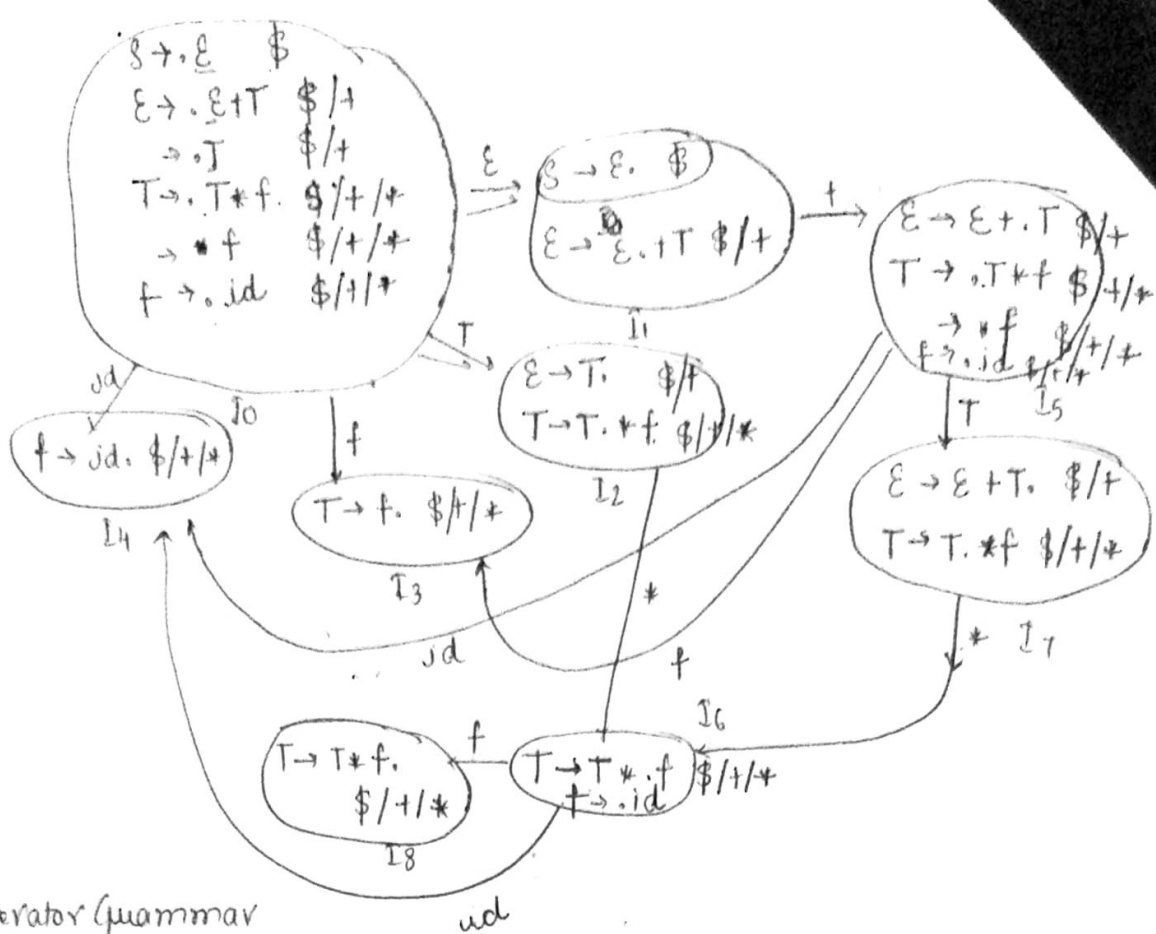
$\rightarrow$  SR conflict  
 $\rightarrow$  it will ignore reduction in case of SR conflict

CLR(1)

$E \rightarrow E + T \mid T$   
 $T \rightarrow T * f \mid f$   
 $f \rightarrow id$

Closure( $S \rightarrow E \quad \$$ )

$\Rightarrow P.T.O \rightarrow$



OPP → Operator Grammar  
 no null production  
 no 2 consecutive non-terminal / ~~grammar~~ variable in RHS of prod

Eg →  $A \rightarrow BC$   
 $B \rightarrow A$  Not a operator Grammar  
 $C \rightarrow C$

$S \rightarrow AaB$   
 $A \rightarrow aA/b$  ~~not~~ operator Grammar  
 $B \rightarrow bB/c$

$E \rightarrow EAE/id$  not a operator Grammar  
 $A \rightarrow +|-/*$

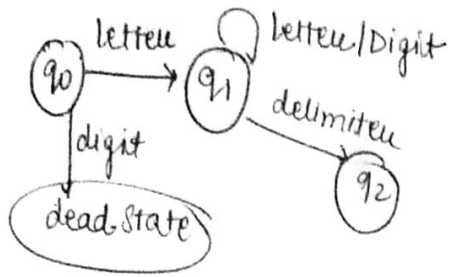
$E \rightarrow E+E | E-E | E * E / id$  operator Grammar

$S \rightarrow SAB/a$  not a operator Grammar  
 $A \rightarrow bBb/b$

$\downarrow$   
 $S \rightarrow SbSbS / SbS/a$

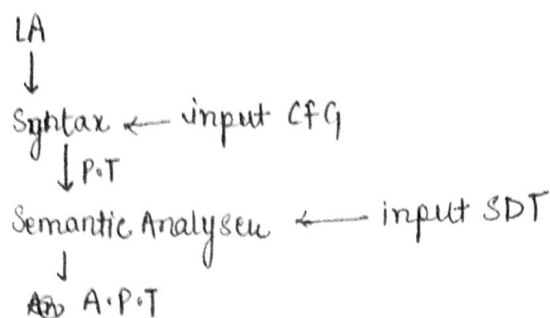
$S \rightarrow AaB$   
 $A \rightarrow aA$  yes it is operator grammar  
 $B \rightarrow bB/c$

PDA are used to design parsers or syntax analysers. And parser verifies syntax of text



Lecture 19

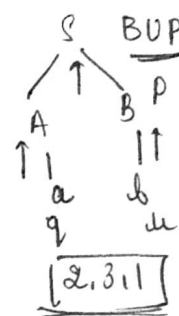
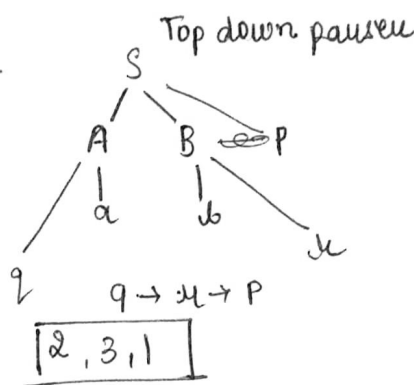
SDT  
Syntax directed translation



CFG + Semantic Action / Semantic Rule → SDT

$S \rightarrow AB$   
 $A \rightarrow a$   
 $B \rightarrow b$   
CFG  
 $\{ \text{printf}(1) \} \rightarrow p$   
 $\{ \text{printf}(2) \} \rightarrow q$   
 $\{ \text{printf}(3) \} \rightarrow u$   
Semantic Action

$w = ab$



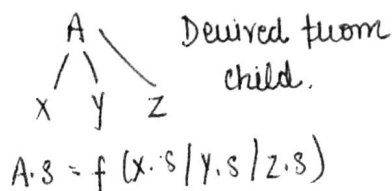
Application of SDT

1. Algebraic expression evaluation
2. Conversion infix to postfix (SDT construction)
3. Construct an SDT to convert infix to prefix exp.
4. " " " " " infix " infix exp.
5. Construct an SDT to convert parse tree to syntax tree.
6. " " " " " Binary number to Decimal number.
7. " " " " " store data in symbol table.

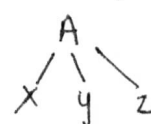
Type of Attributes →

- ① Synthesize attribute
- ② Inherited attribute.

$A \rightarrow XYZ$



$A \rightarrow XYZ$



$X.I = f(A.I / Y.I / Z.I)$

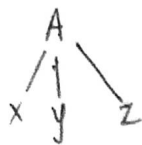
where Derived either from parent or sibling

$2+3*4$

## Type of SDT Attribute

### S attribute

- ↳ uses only synthesized attribute.



$$A.S = f(x.S / y.S / z.S)$$

### L attribute

- ↳ uses both synthesized attribute and inherited attribute with restriction that inherited attribute can inherit value from left sibling

$$y.S = f(A.S / x.S)$$

Q.

- ↳ action part at right most place

$$S \rightarrow AB \{ Pf(1) \}$$

- ↳ It uses both Bottom up and top down approach

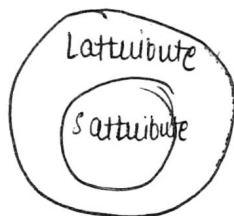
- ↳ Action part can written anywhere in RHS

$$S \rightarrow \{ Pf(1) \} AB$$

$$S \rightarrow A \{ Pf(1) \} B$$

$$S \rightarrow AB \{ Pf(1) \}$$

- ↳ Depth first search



Every S attribute is L attribute

Eg:  $S_1: A \rightarrow BC \{ A.val = B.val + C.val \} \rightarrow$  synthesize att.

$S_2: B \rightarrow DE \{ B.val = D.val * E.val \text{ and } D.val = E.val \}$

↓  
Synthesize att.

↓  
Right inherited att.

∴  $S_1$  is both S & L

$S_2$  is not L

∴ option (3)  $S_1 \rightarrow L$  &  $S_2$  is not L

## Application 1 ★★

1. Construct an SDT for algebraic expression evaluation

P.T.O →

I/p  $\rightarrow 2+3*4$  O/p = 14

Grammar by Bottom up Parser

$E \rightarrow E+T / T$

$T \rightarrow T*f / f$

$f \rightarrow id$

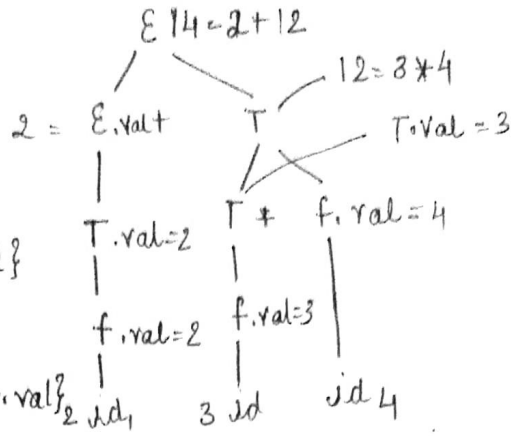
$E \rightarrow E+T \quad \{ E.val = E.val + T.val \}$

$\rightarrow T \quad \{ E.val = T.val \}$

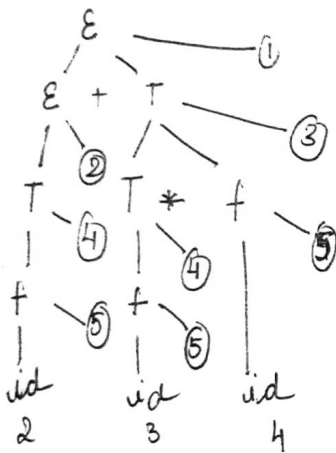
$T \rightarrow T*f \quad \{ T.val = T.val * f.val \}$

$\rightarrow f \quad \{ T.val = f.val \}$

$f \rightarrow id \quad \{ f.val = id.val \}$



By Top Down Parser



Starting from left to right

$\textcircled{5} \rightarrow \textcircled{4} \rightarrow \textcircled{2} \rightarrow \textcircled{5} \rightarrow \textcircled{4} \xrightarrow{*} \textcircled{4} \rightarrow \textcircled{3}$   
 $id=2 \quad f=f \quad E=T \quad id=3 \quad T=f \quad f=id \quad id=12$   
 $f=2 \quad f=3 \quad =4$

$E \leftarrow E+T$   
 $E=14 \quad 2+12$

lecture -20

SDT

$\hookrightarrow Cfg + \text{Semantic Rule}$

$S \rightarrow AB \quad \{ P_f(\psi) \}$   
 $Cfg \quad \text{Action}$

SDT is also called SDD + Translation Scheme

Syntax Directed Definition

Specification

How do evaluate

what are attribute + translation position

$2+3*4$

$\downarrow$

LA

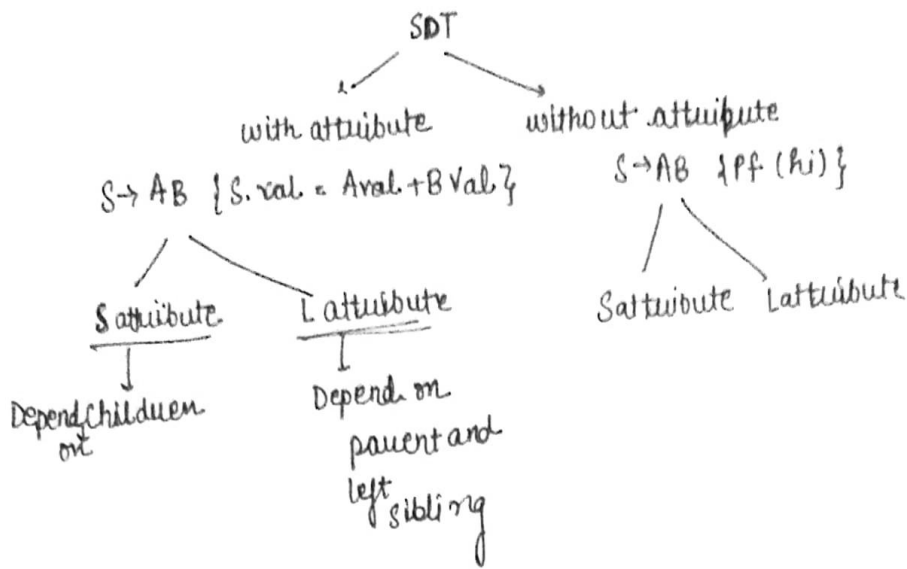
$\downarrow$

SA

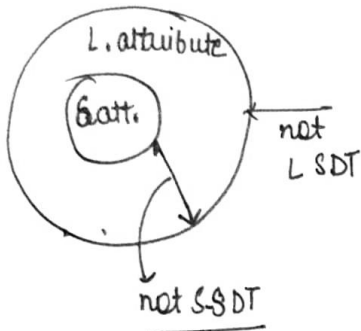
$\downarrow$  PT

Sem A  $\rightarrow$  Type checking

Type casting  
Variable decl.



$A \rightarrow a \{A.x = a.val\}$  where  $x$  is S attribute  
 $S - SDT \rightarrow L - SDT$



Eg 2  $S \rightarrow AB \{B.x = A.y\} \rightarrow x \rightarrow \text{inherited}$   
 $A \rightarrow a \{A.y = a.val\} \rightarrow S - SDT : y$   
 $B \rightarrow b \{B.z = b.val\} \rightarrow S - SDT : z$

} L attribute but not S attribute SDT

Eg 3  $S \rightarrow AB \{A.x = B.val\} \rightarrow x \rightarrow \text{inherited}$   
 $A \rightarrow a \{B.y = a.val\} \rightarrow y \rightarrow \text{synthesized}$   
 $B \rightarrow b \{B.z = b.val\} \rightarrow z \rightarrow \text{synthesized}$

} Not L attribute  
Not S attribute SDT

Eg 4  $E \rightarrow \{Pf(1)\} E + T$  } L attribute SDT  
 $E \rightarrow id \{Pf(id)\}$  } L attribute SDT

$\downarrow$   
 leftmost      rightmost

5).  $S \rightarrow AB \{pf(1)\} - ①$   
 $A \rightarrow a \{pf(2)\} - ②$   
 $B \rightarrow b \{pf(3)\} - ③$

} S attribute SDT  $\rightarrow$  LSDT

- ① if it is LST we will perform Depth first search with left to right scanning
- ② Topological order
- ③ Attribute evaluation

### Attribute

we can apply ~~RRMD~~ BOP and perform RRMD or we can apply Depth Left to Right

if nothing is given without attribute

Then apply Depth scan Left to Right

$S \rightarrow AB \{PF(1)\}$   
 $A \rightarrow a \{PF(2)\}$   
 $B \rightarrow b \{PF(3)\}$

S-SDT as well LST

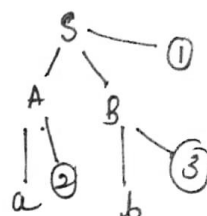
1) Attribute SDT - Depth Left to Right

2) LMD

3) Reverse of LMD

4) RMD

5) RRMD



2) S  
    1  
    AB    for 2)  $1 \rightarrow 2 \rightarrow 3$   
    1  
    aB  
    1  
    ab

3) RLMD

$3 \rightarrow 2 \rightarrow 1$

4) S  
    1  
    AB  
    1  
    Ab  
    1  
    ab

1)  $2 \rightarrow 3 \rightarrow 1$   
 2)  ~~$2 \rightarrow 3 \rightarrow 1$~~   
 So it will be  
 $1 \rightarrow 3 \rightarrow 2$   
 5) RRMD  
 $2 \rightarrow 3 \rightarrow 1$

Lecture 21

P.T.O. →

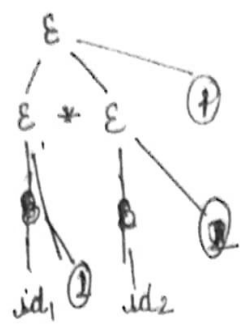
Ques  $E \rightarrow E * E$  {pf (+)} - ①  
 $E \rightarrow id$  {pf ("id")} - ② Shift Red SGT

$w = id_1 * id_2$

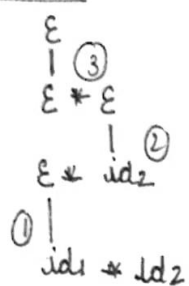
The output will be

2, 2, 1

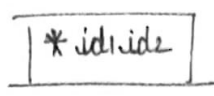
Translation order  
id<sub>1</sub>



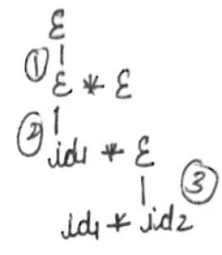
Using RRMD



Using IDP



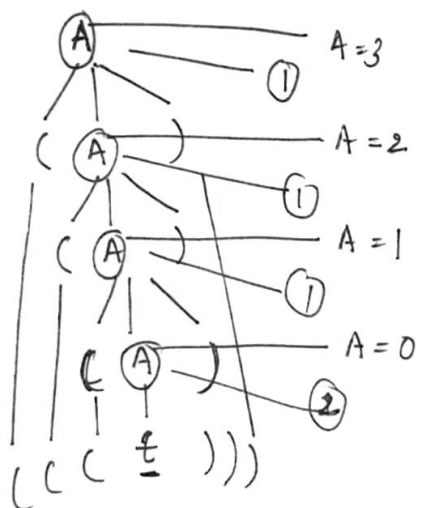
we will print  
id1 id2 \*



Ques Construct an SDT for Balanced Parenthesis

I/p  $\rightarrow ( ( ( ) ) )$   
o/p = 3

to Count Balanced  
Grammar  $\Rightarrow A \rightarrow (A) \{A.count = A.count + 1\}$  - ①  
 $A \rightarrow \epsilon \{A.count = 0\}$  - ②

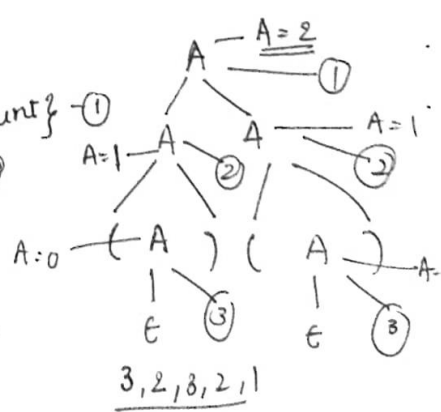


$\therefore$  final ans will be  
A=3

I/p = ( ) ( )

$A \rightarrow AA \{A.count = A.count + A.count\}$  - ①  
 $A \rightarrow (A) \{A.count = A.count + 1\}$  - ②  
 $A \rightarrow \epsilon \{A.count = 0\}$  - ③

Final A = 2



Que Construct an SDT to convert from infix to postfix notation

postfix = 2 3 4 \* +

I/p = 2+3\*4

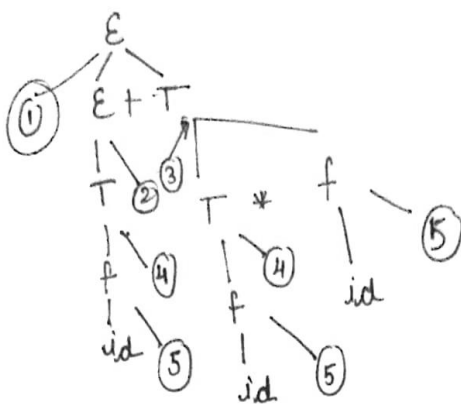
O/p = 2 3 4 \* +

Grammar -  $\epsilon \rightarrow \epsilon + T$  {print "("} ①  
 $\rightarrow T$  { } ②  
 $T \rightarrow T * f$  {print "("} ③  
 $\rightarrow f$  { } ④  
 $f \rightarrow id$  {print "("} ⑤

} d. attribute SDT

∴ final ans

tid<sub>1</sub> + id<sub>2</sub> id<sub>3</sub>



\*\*\*

for postfix

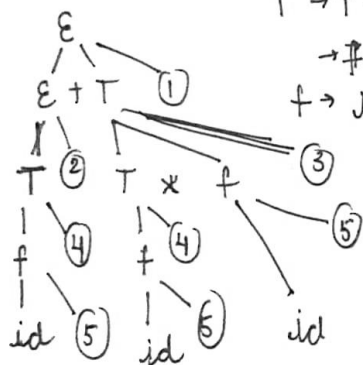
I/p = 2+3\*4

O/p = 2 3 4 \* +

1 → 5 → 4 → 2 → 3 → 5 → 4 → 5

Grammar

$\epsilon \rightarrow \epsilon + T$  {print "("}  
 $\rightarrow T$  { }  
 $T \rightarrow T * f$  {print "("}  
 $\rightarrow f$  { }  
 $f \rightarrow id$  {print "("}



5 → 4 → 2 → 5 → 4 → 5 → 3 → 1

id<sub>1</sub> id<sub>2</sub> id<sub>3</sub> \* +

final ans

with attributes of statements

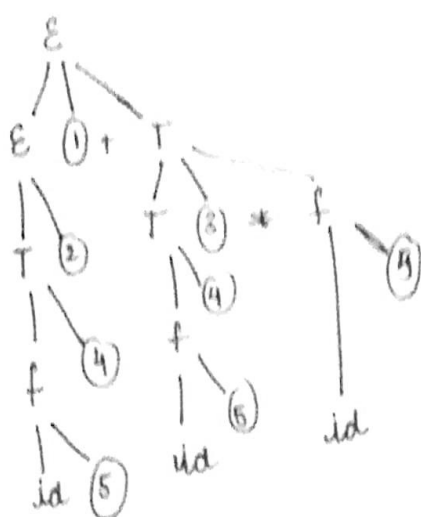
{ Eval = Eval Tval + }  
 { Eval = Tval }  
 { Tval = Tval fval \* }  
 { Tval = fval }  
 { fval = id.val }

eg

$\epsilon \rightarrow \epsilon$  {pf + } + T  
 $\rightarrow T$  { }  
 $T \rightarrow T$  {pf + } \* f  
 $\rightarrow f$  { }  
 $f \rightarrow id$  {pf id }

I/p = 2+3\*4

O/p = ?



O/p  $\rightarrow 5 \rightarrow 4 \rightarrow 2 \rightarrow 1 \rightarrow 5 \rightarrow 4 \rightarrow 3 \rightarrow 5$

$id_1 + id_2 * id_3$   
 $[ 2 + 3 * 4 ]$   
 { Infix to Infix }  
 \*\*\*

## Lecture - 22

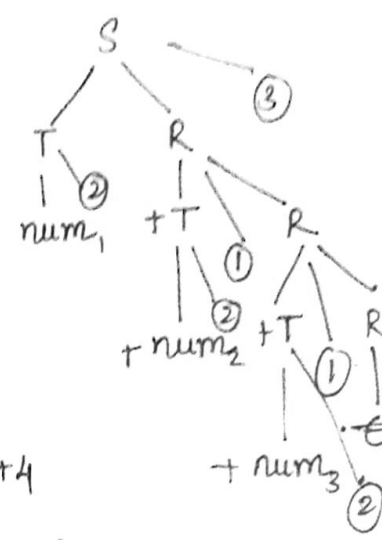
Que 1  $S \rightarrow TR$  (3) I/p =  $9 + 5 + 2$

$R \rightarrow +T \{ \text{print}(' + '); \} R / \epsilon$  (1) O/p = ?

$T \rightarrow \text{num.} \{ \text{print}(\text{num.val}); \}$  (2)

$2 \rightarrow 2 \rightarrow 1 \rightarrow 2 \rightarrow 1 \rightarrow 3$

9 5 + 2 + final output



Que 2  $S \rightarrow ER$

$R \rightarrow *E \{ \text{print}('*'); \} R / \epsilon$  (1)

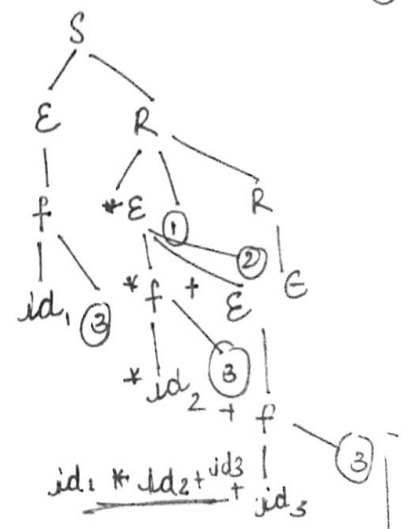
$E \rightarrow f + E \{ \text{print}('+'); \} / f$  (2)

$f \rightarrow (s) / \text{id} \{ \text{print}(\text{id.value}); \}$  (3)

I/p =  $2 * 3 + 4$

$3 \rightarrow 3 \rightarrow 3 \rightarrow 2 \rightarrow 1$

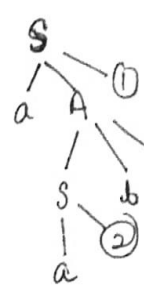
2 3 4 + \* final output



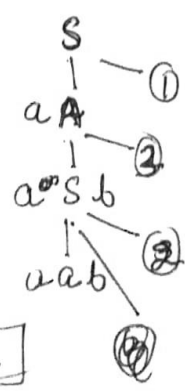
Que 3  $S \rightarrow aA \{ \text{print} 1 \}$  I/p = aab

$S \rightarrow a \{ \text{print} 2 \}$

$A \rightarrow Sb \{ \text{print} 3 \}$



TDP = 132



$2 \rightarrow 3 \rightarrow 1$

final output

2 3 1

Q4  $A \rightarrow PQ$   $A \rightarrow XY$

Rule 1  $P.i = A.i + 2$

$Q.i = P.i + A.i$  &  $A.s = P.s + Q.s$

} L attribute

Rule 2  $X.i = A.i + Y$  &  $Y.i = X.s + A.i$

} Not L attribute also Not S

Only Rule 1 is L attributed

Que 5  $E \rightarrow E_1 \# T \{E.val = E_1.val * T.val\}$  - (1)

$\rightarrow T \{E.value = T.val\}$  - (2)

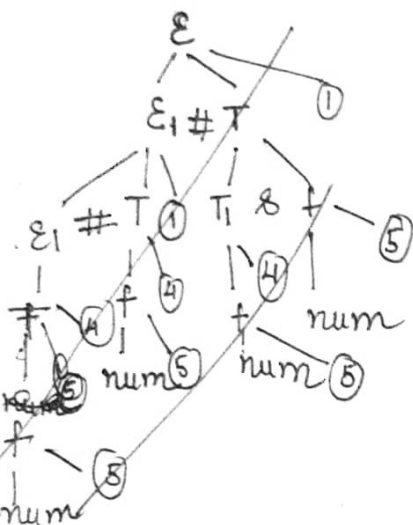
$Ip = 2 \# 3 \# 5 \# 6 \# 4$

$T \rightarrow T_1 \& f \{T.val = T_1.value + f.value\}$  - (3)

$O/p = ?$

$\rightarrow f \{T.value = f.value\}$  - (4)

$f \rightarrow num \{f.value = num.value\}$  - (5)



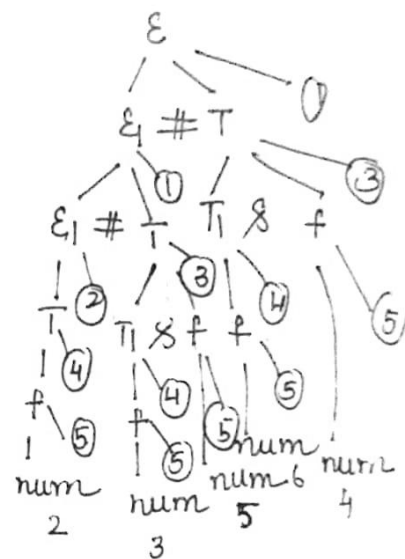
$5 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 5 \rightarrow 4$

$5 \rightarrow 4 \rightarrow 2 \rightarrow 5 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$   
 $\rightarrow 5 \rightarrow 4 \rightarrow 5 \rightarrow 3 \rightarrow 1$

$2 * 8 = 16$

$10 \times 16 = 160$

final ans



Que 6 Operator Grammar

$P \rightarrow QR$   $\rightarrow$  This violates OG as no <sup>non</sup> 2<sup>nd</sup> terminal should be adjacent

$P \rightarrow QSR$

$P \rightarrow \epsilon$   $\rightarrow$  not allowed

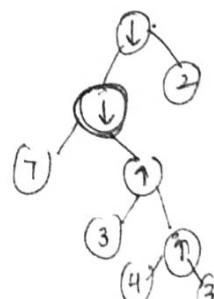
$P \rightarrow QTR$

Que 7 Consider 2 binary operators  $\uparrow$  and  $\downarrow$  with the precedence of operators being lower than that of the  $\uparrow$  operator

$\uparrow \rightarrow$  right Associative

$7 \downarrow 3 \uparrow 4 \uparrow 3 \downarrow 2$

$\downarrow \rightarrow$  left Associative



Que In a Bottom up evaluation of a syntax directed definition inherited attribute can be evaluated

→ only if definition has synthesized attribute

Que

$S \rightarrow a \{ \text{print } 1 \} aA$  - (1)

$S \rightarrow c \{ \text{print } 2 \}$  - (2)

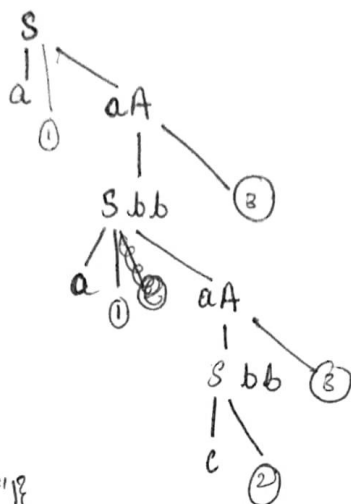
$A \rightarrow sbb \{ \text{print } 3 \}$  - (3)

} Attribute

if = aaaaacbbbb

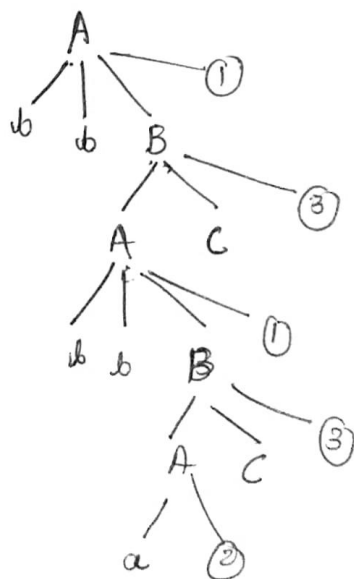
o/p = 23131

BUP


$$2 \rightarrow 3 \rightarrow 1 \rightarrow 3 \rightarrow 1$$

using L attribute eval  
 $1 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 3$

Que  $A \rightarrow bbb \mid \text{printf}(" ")$   $\text{I/p} = bbb bace$   $\text{O/p} =$   
 $A \rightarrow a \mid \text{pf}("x")$   
 $B \rightarrow AC \mid \text{pf}("-")$

$$2 \rightarrow 3 \rightarrow 1 \rightarrow 3 \rightarrow 1$$


Que

$S \rightarrow AS \{p \{ "1" \} \}$

$S \rightarrow AB \quad " \quad 2$

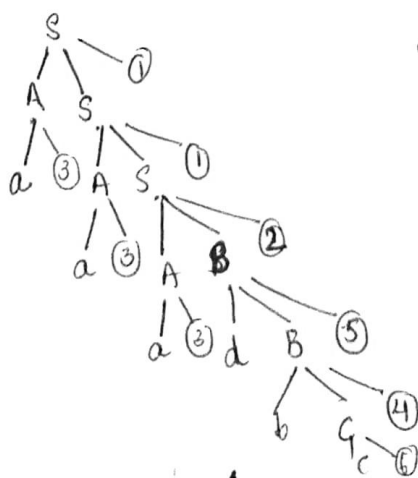
$A \rightarrow a \quad " \quad 3$

$B \rightarrow bc \quad " \quad 4$

$B \rightarrow dB \quad " \quad 5$

$C \rightarrow c \quad " \quad 6$

$w = a a a d b c$


$$3 \rightarrow 3 \rightarrow 3 \xrightarrow{6.5} 4 \rightarrow 5 \rightarrow 2 \rightarrow 1$$

~~3334521~~

333645211

Construct an SDR to convert given Binary no into decimal.

Binary no: 101

Decimal no: 5

$$\begin{array}{r} 1 \quad 0 \quad 1 \\ \hline 101 \\ 2 \times L + B = 2 \times 2 + 1 = 5 \end{array}$$

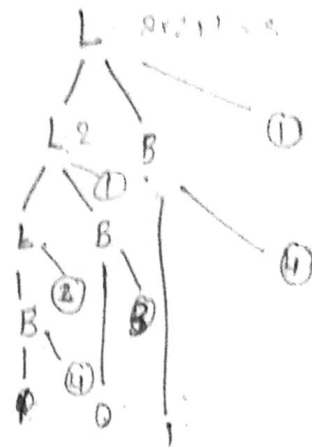
$$\begin{array}{r} L \quad B \\ \hline 1010 \\ 2 \times 5 + 0 = 10 \end{array}$$

$L \rightarrow LB \quad \{ L.dval = 2 * L.dval + B.dval \}$

$\rightarrow B \quad \{ L.dval = B.dval \}$

$B \rightarrow 0 \quad \{ B.dval = 0 \}$

$\rightarrow 1 \quad \{ B.dval = 1 \}$



# No of Bit Counting

$4 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow 4 \rightarrow 1$

$L \rightarrow LB \quad \{ L.x = L.x + B.x \}$

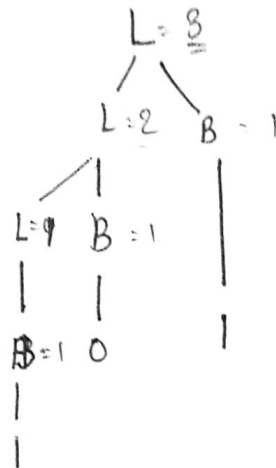
$\rightarrow B \quad \{ L.x = B.x \}$

$B \rightarrow 0 \quad \{ B.x = 1 \}$

$\rightarrow 1 \quad \{ B.x = 1 \}$

Ip = 101

O/p  $\Rightarrow 2 = 3$



Ip = 101.101  $\rightarrow \frac{5}{8} = 0.625$

O/p  $\Rightarrow 5 + 0.625$

= 5.625

$\Rightarrow$  Quotient  $\rightarrow B \rightarrow L.L \quad \{ B.dval = L.dval + L2.dval / 2^{L2.nval} \}$   
 $L \rightarrow LB \quad \{ L.nval = L.nval + B.nval, L.dval = 2 * L.dval + B.dval \}$   
 $\rightarrow B \quad \{ L.nval = B.nval, L.dval = B.dval \}$   
 $B \rightarrow 0 \quad \{ B.nval = 0, B.dval = 0 \}$   
 $\rightarrow 1 \quad \{ B.nval = 1, B.dval = 1 \}$

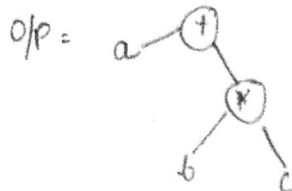
Tree  $\rightarrow$



## Syntax Tree

→ all nodes are terminal

I/p =  $a + b * c$



## Parse Tree (abstract syntax tree)

→ nodes can be both terminal or non terminal

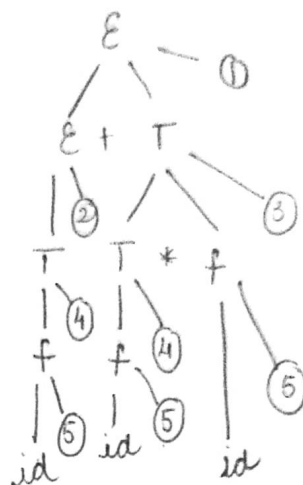
$E \rightarrow E + T \quad \{E.ptu = mknode(E.ptu, +, T.ptu)\}$

$\rightarrow T \quad \{E.ptu = T.ptu\}$

$T \rightarrow T * f \quad \{T.ptu = mknode(T.ptu, *, f.ptu)\}$

$\rightarrow f \quad \{T.ptu = f.ptu\}$

$f \rightarrow id \quad \{f.ptu = mknode(null, id, null)\}$



## Intermediate Code Generator

### Intermediate Code

Linear form

Nonlinear form (Tree)

Postfix

3 Address Code

Syntax Tree

Directed

Acyclic

Graph (DAG)

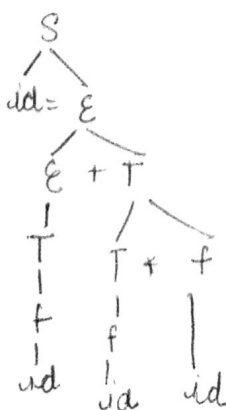
Static Single Assignment (SSA)

$S \rightarrow id = E;$

$E \rightarrow E + T / T$

$T \rightarrow T * f / f$

$f \rightarrow id$



Code

↓ LA

Stream of Token

↓ SA

Parse Tree

↓ Sem A

SVT (Decorated Parse Tree)

↓ ICG

Intermediate Code / Representation

Machine Independent Code

ICG → Postfix

→ 3 Address Code

→ SSA

→ Syntax tree

→ DAG

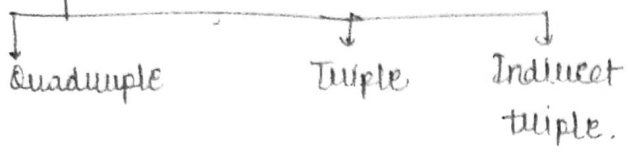
# Postfix DAG

$$x = a + b * c;$$

$$xabc**+=$$

Optimized postfix  $abc**+$

## 3 add. code



3 add. code: In a single line we have

at most 3 address

$$t1 = b * c;$$

Three variable address

$$+z = a + t1;$$

$$x = +z;$$

Optimize

$$t1 = b * c;$$

$$x = a + t1;$$

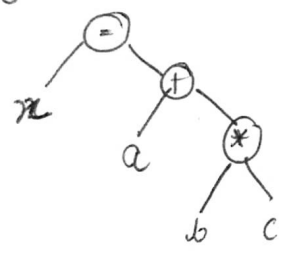
Further optimize

$$b = b * c;$$

$$a = a + b;$$

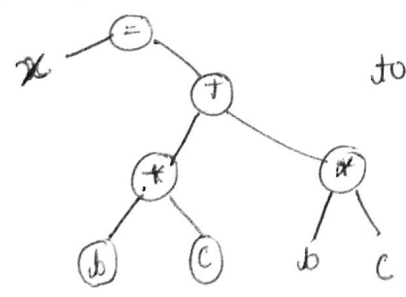
## Syntax tree

$$x = a + b * c$$

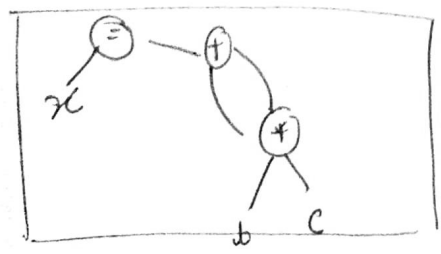


DAG : to eliminate common subexpression

$$x = b * c + b * c$$



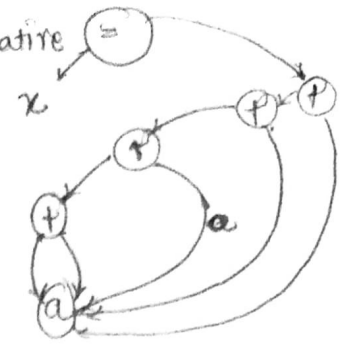
to reduce the repetition of  $b * c$



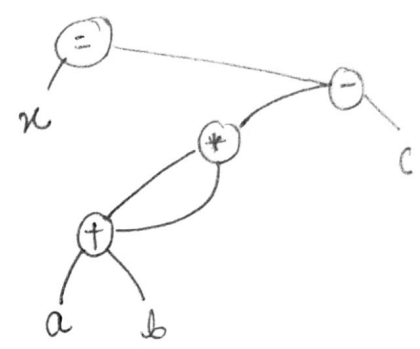
$$x = a + a + a + a + a$$

acc to Rule

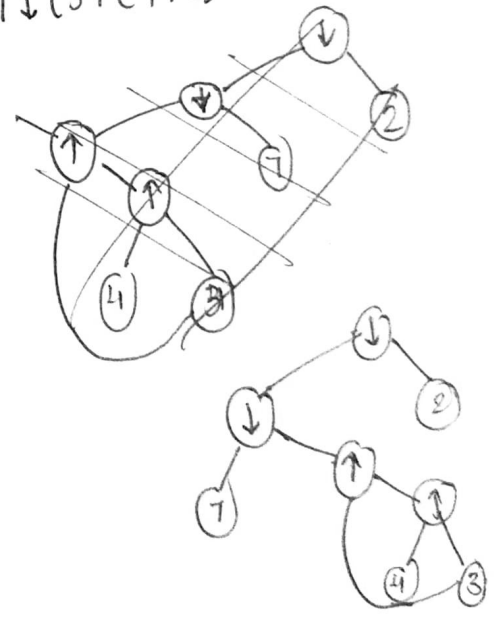
(+) is left associative



$$x = (a + b) * (a + b) - c$$



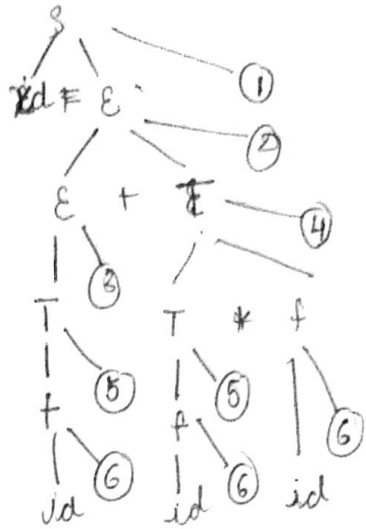
$$7 \downarrow (3 \uparrow (4 \uparrow 3)) \downarrow 2$$



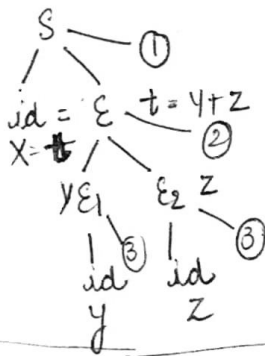
3 add code

$$S \rightarrow \text{id} = \varepsilon \quad \& \text{gen}(\text{id.val}) = \varepsilon.\text{val}\}$$
$$E \rightarrow E + F \quad \begin{cases} j2 = \text{new temp}() \\ \text{gen}(j2) = F, \text{val} \end{cases}$$
$$E \rightarrow E + E \quad \text{gen}(t_2 = E_1.\text{val} + T_1.\text{val}) \quad E.\text{val} = t_2$$
$$\rightarrow T \quad \{E.val = T.val\}$$
$$T \rightarrow T * f \quad \left\{ \begin{array}{l} t1 = \text{new-temp}() \\ \text{gen}(t1, T1.\text{val} * f.\text{val}) \quad T.\text{val} = t1 \end{array} \right\}$$
$$\rightarrow f \text{ d } T, \text{ val } f, \text{ val } \}$$
$$f \rightarrow id \quad \{f.val = id.val\}$$

~~6 → 5 → 3 → 6 → 5 → 6 → 4 → 2 → 1~~

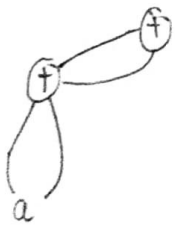
One
$$S \rightarrow id = \epsilon \quad \{ gen(id.place = \epsilon.place) \} \quad - (1)$$
$$E \rightarrow E_1 \mid E_2 \quad \{ t = \text{newtemp}(); \text{gen}(t = E_1.\text{place} \mid E_2.\text{place}); E.\text{place} = t \} - \textcircled{2}$$
$$\varepsilon \rightarrow \text{id} \{ \varepsilon, \text{place} = \text{id} \cdot \text{place} \} \quad - (3)$$
$$\underline{\underline{x = y + z}}$$
$$3 \rightarrow 3 \rightarrow 2 \rightarrow 1$$

option B

$$\begin{aligned} X &= t \\ Y &= t \end{aligned}$$


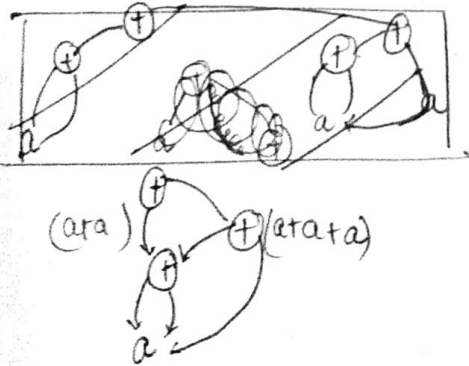
due

Construct DAG for  $(a + a) \neq (a + a)$



no of nodes = 3

Ques Construct DAG for  $(a+a)+(a+a+a)$



Ques 3 address Code - In a statement

we have at most 3 address

1.  $x = y \text{ op } z$
2.  $x = \text{op } y$
3.  $x = y$
4.  $x = *y$
5.  $x = \&y$
6.  $x = a[i] \text{ } (*a+i)$
7.  $a[i] = x$
8. goto L (Unconditional Jump)
9. if  $x < y$  goto L (conditional Jump)
10.  $y = f(x_1, x_2, \dots, x_n)$  ~~X~~
11.  $x = a[i, j]$  ~~X~~ not  
3 add  
code

Ques Construct 3 add code for the following C statement

```
if (a > b)
    print("hi");
else
    print("bye");
```

```
100. if (a > b) goto 103
101. print("bye");
102. goto 104
103. print("hi");
104. Exit/End
```

Ques if (a > b) then t = 1 else s = 0

```
i: if (a > b) goto i+3
i+1: s = 0
i+2: goto i+4
i+3: t = 1
i+4: Exit/End
```

Ques if a < b and c < d then t = 1 else s = 0

```
i: if a < b goto i+2
i+1: goto i+3
i+2: if c < d goto i+5
i+3: s = 0
i+4: goto i+5
i+5: t = 1
i+6: Exit/End
```

Ques for (i = 0; i < 10; i++)

```
{
    x = a + b * c;
}
```

```
100. i = 0
101. if (i < 10) goto 103
102. goto 108
103. t1 = b * c;
104. t2 = a + t1;
105. x = t2;
106. i = i + 1;
107. goto 101
108. Exit
```

Quadruple - we have 4 columns

$x = a + b * c;$

3 add code req: →

1. Quadruple
2. triple
3. Indirect triple

t1 = b \* c;

t2 = a + t1;

x = t2;

| result | op1 | op2 | operator |
|--------|-----|-----|----------|
| t1     | b   | c   | *        |
| t2     | a   | t1  | +        |
| x      | -   | t2  | =        |

Advantage: → ① reuse the values.  
② Time saving.

Disadvantage ① takes more space in form of result column

Triple - we have 5 columns

|            | operand1 | operand2 | operator |   |
|------------|----------|----------|----------|---|
| <u>add</u> | 1000     | b        | *        |   |
|            | 2000     | a        | 1000     | + |
|            | 3000     | -        | 2000     | = |

Advantage - save space

Disadvantage - the computation of result  
② more time consumption

## Indirect Triple

We can save the result address in another address to improve mobility

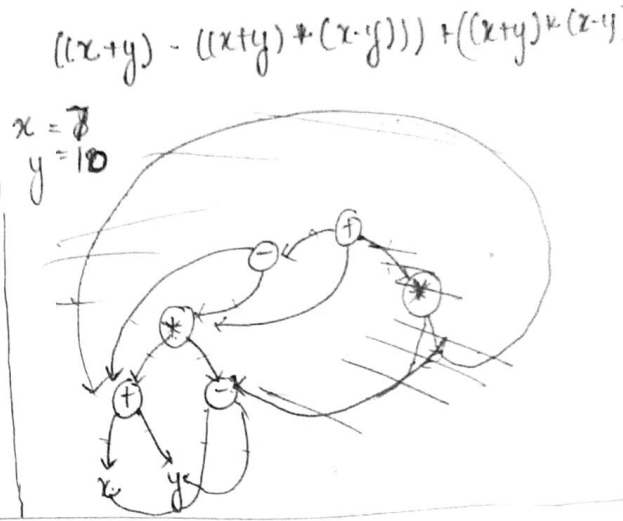
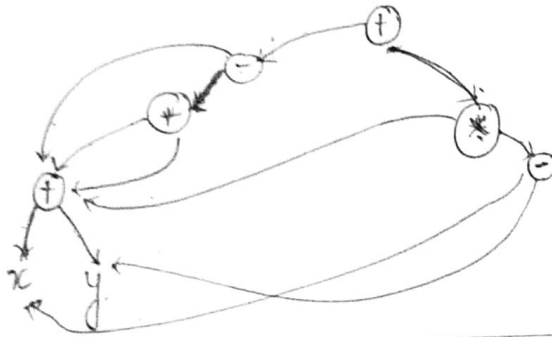
|      |      |
|------|------|
| 5000 | 1000 |
| 6000 | 2000 |
| 7000 | 3000 |

less space

## Lecture 27

Construct a DAG for following statement and find  $x$  &  $y$  where  $x$  is no of nodes and  $y$  is no of edges.

$$((x+y) - ((x+y) * (x+y))) + ((x+y) * (x-y))$$



Consider 3 add code  $\rightarrow$  find out min no of temp variable in optimized add code

$t1 = a+b;$   
 $t2 = c+d;$   
 $t3 = t1+t2;$   
 $t4 = t2+t3;$   
 $t5 = t4+t3;$

$a = a+b;$   
 $c = c+d;$   
 $t1 = a*c;$   
 ~~$t2 = t1+c;$~~   
 ~~$t3 = t1+t2;$~~

min = 2 var (temp)

$a = a+b;$  total var = 6  
 $c = c+d;$   
 $t1 = a*c;$   
 $c = c+t1;$   
 $t2 = t1+c;$

SSA - variable should be defined only once in LHS

Que Consider  $\rightarrow$

$x_1 = u-v;$   
 $y_1 = x_1 * v;$   
 $x_2 = y_1 + w;$   
 $y_2 = t-z;$   
 $y_3 = x_2 * y_2;$

(min no var used to convert it into SSA)

total distinct var = 10

$x_1, x_2, y_1, y_2, y_3, u, t, v, w, z$

Que SSA

$$\begin{aligned} p &= a-b & p_1 &= a-b \\ q &= p * c & \Rightarrow q_1 &= p_1 * c & \text{Option} \\ p &= u * v & p_2 &= u * v & \underline{B} \\ q &= p + q & q_2 &= p_2 + q_1 \end{aligned}$$

Que  $x = (a+b) * (a+b) + c$

$$\left. \begin{aligned} t &= a+b \\ u &= t * t \\ x &= t + u \end{aligned} \right\} \begin{aligned} t &= a+b \\ u &= t * t \\ x &= u + c \end{aligned} \Rightarrow \begin{aligned} a &= a+b \\ a &= a * a \\ x &= a + c \end{aligned}$$

$$\begin{aligned} t_1 &= a+b \\ t_2 &= a+b \\ t_3 &= t_1 * t_2 \\ t_4 &= t_3 + c \\ x &= t_4 \end{aligned}$$

Que  $x = (y+z) * y + z$

$$\left. \begin{aligned} t &= y+z \\ y &= t * y \\ x &= y + z \end{aligned} \right\} \begin{aligned} &4 \text{ variable} \Rightarrow \\ &y = t * y \\ &y = y + z \end{aligned}$$

Que  $q + u/3 + 5 - t * 5 + u * v/w$

$$\begin{aligned} t_1 &= u/3 \\ t_2 &= v/w \\ t_3 &= u * t_2 \\ t_4 &= t * 5 \\ t_5 &= t_3 + t_4 \end{aligned}$$

$$\begin{aligned} t_5 &= t_3 + t_4 \\ t_6 &= t_5 + 5 \\ t_7 &= t_3 + t_4 \\ t_8 &= t_6 - t_7 \end{aligned}$$

Total 8 temp var  
are required.

Lecture - 28

Code Optimization

Que  $x = 10;$   
 $a = a * x;$   
 $c = c - a;$

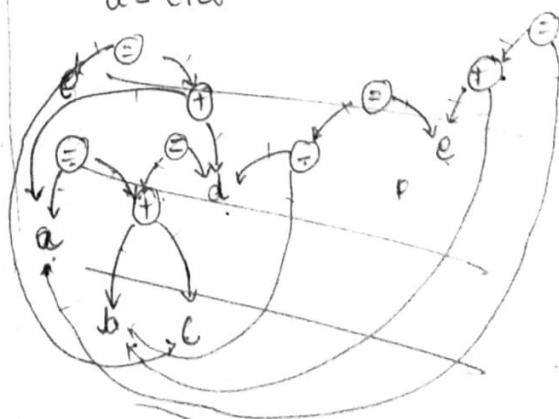
|      | Operand 1      | Op2           | Operator |
|------|----------------|---------------|----------|
| 1000 | <del>x</del> - | <del>10</del> | =        |
| 2000 | a              | 1000          | *        |
| 3000 | c              | 2000          | -        |

Indirect tripple

|      |      |
|------|------|
| 5000 | 1000 |
| 6000 | 2000 |
| 7000 | 3000 |

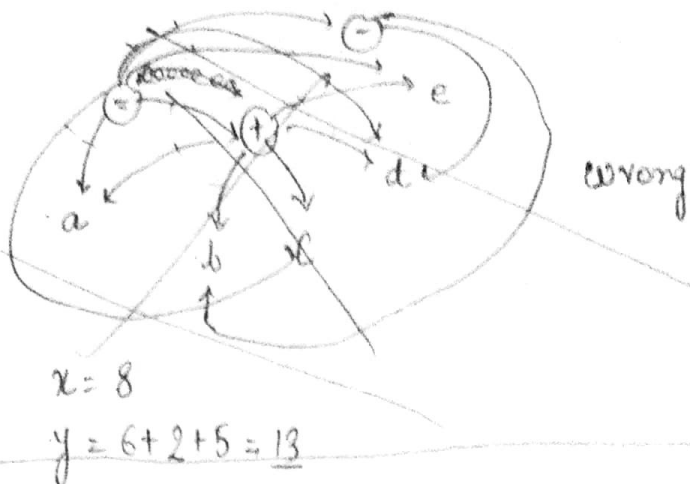
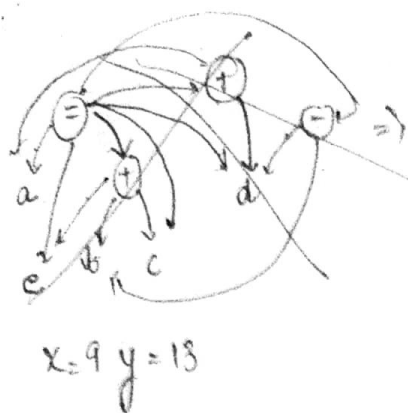
Que DAG  
 $a = b + c$   
 $c = a + d$   
 $d = b + c$   
 $e = d - b$   
 $a = e + b$

$x = ?$  vertex  
 $y = ?$  edge



$$x = 14$$

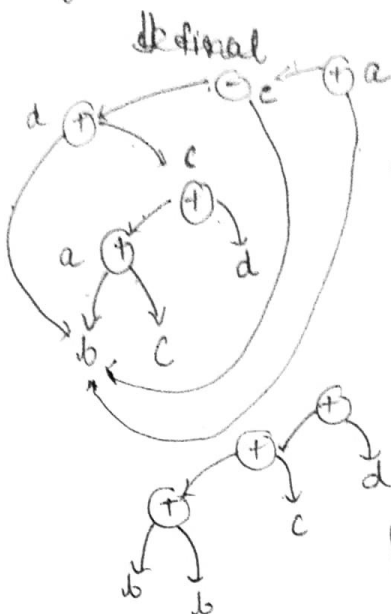
$$y = 18$$



$$\begin{aligned} a &= b+c \\ c &= a+d \\ d &= b+c \\ e &= d-b \\ a &= e+b \end{aligned}$$

$$\begin{aligned} a &= d \\ e &= c \end{aligned}$$

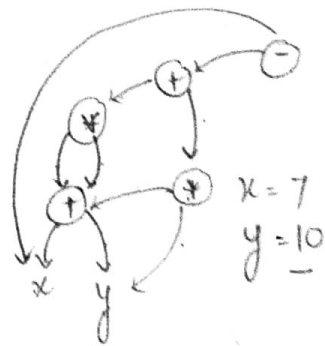
$$\begin{aligned} a &= e+b \\ a &= d-b+b \\ a &= d \\ a &= b+c \\ a &= b+ard \\ a &= \underline{\underline{b+b+c+d}} \end{aligned}$$

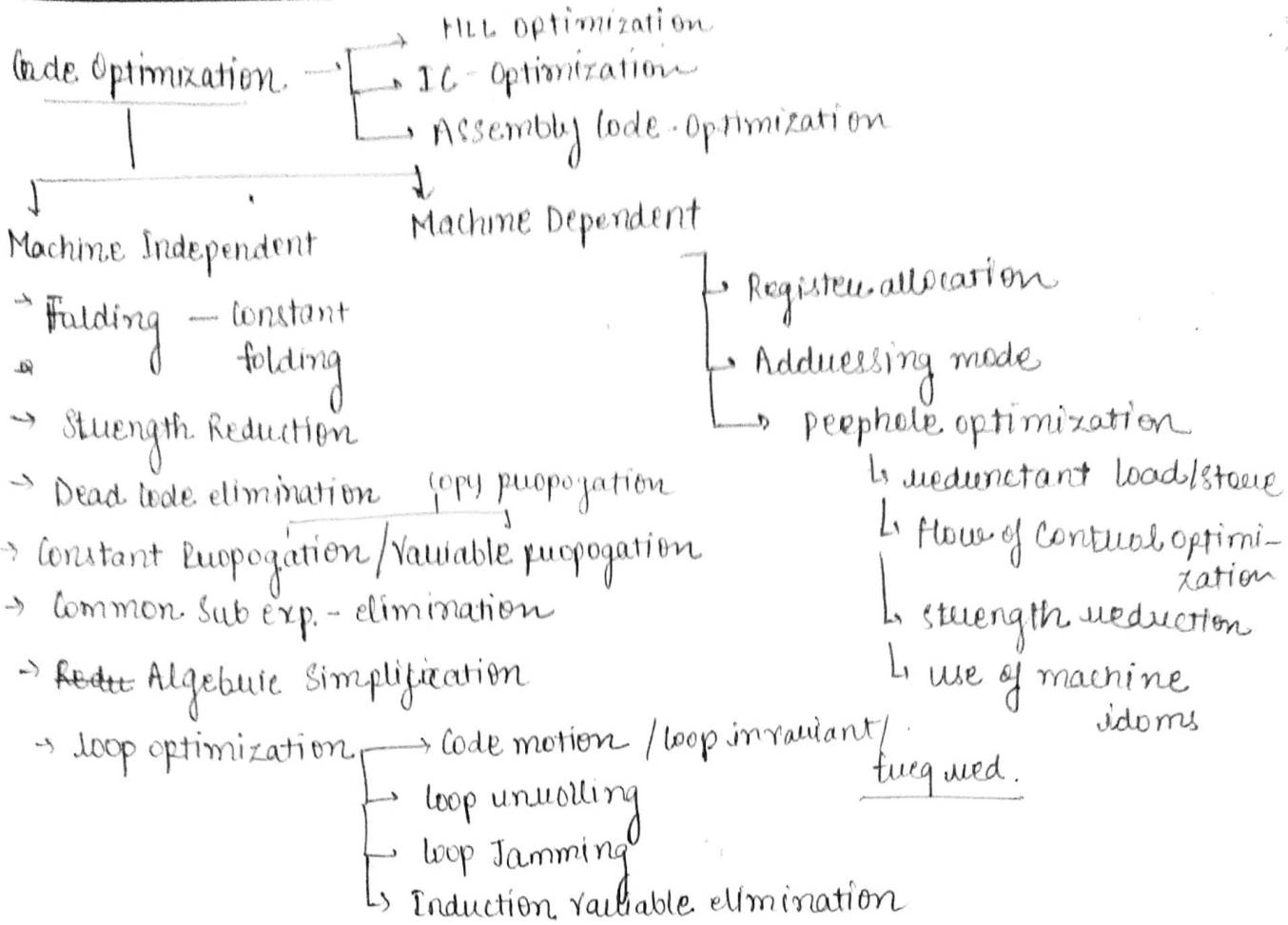


Construct DAG for full exp

$$((x+y) * (x+y)) + (y + (x+y)) - x$$

Construct DAG and find min no of node and edges

$$a = (x+y) + (x+y) + (x+y) + x$$




Constant folding → Constant ~~is~~ folded ~~into~~ ~~code~~

①  $x = 4 * 2.14; \Rightarrow x = 8.56;$

②  $a = 3 + 4 * b \Rightarrow$  Can't perform CF

③  $a = 3 + 4 + b \Rightarrow a = 12 + b;$

④  $\text{for}(\text{int } j = 0; j \leq n; j++)$

$\{ \text{Arr}[2 * 3] = 10; \Rightarrow \text{Arr}[6] = 10;$   
 $\}$

due  $x = -y * (a + b)$

$\left. \begin{array}{l} t_1 = a + b \\ t_2 = -y * t_1 \\ x = t_2 \end{array} \right\} \rightarrow \begin{array}{l} t_1 = a + b \\ x = -y * t_1 \end{array} \rightarrow \begin{array}{l} a = a + b \\ y = -y * a \end{array}$

| result | op1 | op2 | operator |
|--------|-----|-----|----------|
| t1     | a   | b   | +        |
| t2     | -y  | t1  | *        |
| x      | t2  | -   | =        |

Quadruple

|      |      |      |   |
|------|------|------|---|
| 1000 | a    | b    | + |
| 2000 | -y   | 1000 | * |
| 3000 | 2000 | -    | = |

Strength reduction - replace higher cost operation to lower cost operation

$$A = 4 * A; \rightarrow A = A + A + A + A; \rightarrow A = A \ll 2$$

↓  
Multiplication is higher cost operation

← is lower cost  
← is again lower  
↓  
left shift

Copy propagation → Constant propagation

int x = 12;

int y = 7 - x/2;

return y \* (24/x + 2);

Don't propagate constant variable

↓ here we'll put value of x

$$y = 7 - 12/2 = 7 - 6 = 1 \leftarrow \text{Constant folding}$$

$$\text{return } y * (24/x + 2); \leftarrow y * (212) = 114 = \underline{5}$$

Eg →

$$\begin{array}{l} a = 5 \\ b = a + c \\ d = a + e \end{array}$$

$$\begin{array}{l} b = 5 + c \\ d = 5 + e \end{array}$$

Variable propagation →

$$\begin{array}{l} a = x \\ b = a + c \\ d = a + e \end{array}$$

$$\begin{array}{l} b = x + c \\ d = x + e \end{array}$$

Eg →

$$\begin{array}{l} a = x \\ b = a + c \\ a = d + e \\ g = a * f \end{array}$$

$$\begin{array}{l} b = x + c \\ a = d + e \end{array}$$

$$g = \cancel{d + e} * f \rightarrow g = a * f$$

Dead code elimination: 1. A = 10

2. B = 20

3. C = 30

4.  $C = A + B * C$  → Dead code not being used anywhere

5. X = A + B

6. Y = X + A

7. Z = X + Y

8. Pf ("Z")

Common Sub Expression Elimination:

$$a = b + c$$

$$d = 2 + 3 + b + c$$

$$a = b + c$$

$$d = 2 + 3 + a$$

$$a = b + c$$

$$d = 5 + a$$

Algebraic Simplification  $\Rightarrow$   $a = a + 0$   
 $a = a \times 1$

## Loop Optimization

1) Code Motion / Loop Invariant / Code movement  $\Rightarrow$

```
while (i < 1000)
```

```
{  
  A = sinx/cosx * j  
}
```

does not depend on loop and is computed every time  
↓  
Should be moved either up ward or downward so that the program does not affect

```
t = sinx/cosx  
while (j < 1000)  
{  
  A = t * j  
}
```

Eg  $\rightarrow$  for (int i = 0; i < n; i++)  
{  
  $\overbrace{a = a * i;}$  inducting variable  
 x = y \* z;  
}

↓  
x = y \* z;  
for (i = 0; i < n; i++)  
{  
 a = a \* i;  
}

2) Loop Unrolling

```
while (j < 100)  
{  
  a[i] = j;  
  j = j + 1;  
}
```

```
while (j < 50)  
{  
  a[i] = j;  
  j = j + 1;  
  a[i] = j;  
  j = j + 1;  
}
```

Eg  $\rightarrow$  for (i = 0; i < 99; i++)  
{  
 printf("Today class");  
}

```
for (i = 0; i < 49; i++)  
{  
  printf("Today class");  
  printf("Today class");  
}
```

3) Loop Jamming :-

```

for (i = 1; i <= n; i++)
{
    x = x + i;
}
for (j = 1; j <= n; j++)
{
    y = y + j;
}

```

```

for (i = 1; i <= n; i++)
{
    x = x + i;
    y = y + i;
}

```

Lecture 30

Loop Optimization

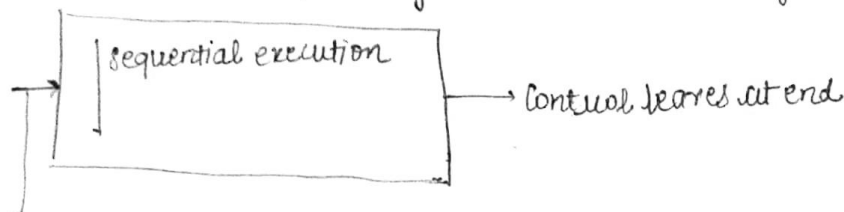
4. Induction Variable

How to identify loop in a program.

Ans 1

using Program flow graph  
or  
Control flow graph

Basic Block :- Sequence of 3 add. code with no jump.



```

x = a + b * c
t1 = b * c
t2 = a + t1
x = t2

```

Control enters at Beg.

1) Set of instructions (3 add c) executed in a seq

# Control never enters in middle and leave at middle

# Basic Block starts from one leader to next leader not including next leader.

How to find Basic Block:

↓

In order to find basic block we need to find leader

↓ what is leader??

1. 1<sup>st</sup> statement is always a leader.
2. Target of conditional or unconditional jump is a leader
3. Statement after conditional or unconditional jump is a leader

int fact (int x)

```

{
  int f = 1;
  for (i = 2; i ≤ x; i++)
  {
    f = f * i;
    return f;
  }
}

```

1.  $f = 1 \rightarrow \text{leader}$

2.  $j = 2$

3. if ( $j \leq x$ ) goto 5.  $\rightarrow \text{leader}$

4. goto 7.  $\rightarrow \text{leader}$

5.  $f_1 = f * j \rightarrow \text{leader}$

6. ~~return~~  $f = f_1$

7. ~~do~~  $j = j + 1$

8. goto 3.

9. return f  $\rightarrow \text{leader}$

# Basic Block.

# Control -

# node -

# edge

Blocks  $B_1 \rightarrow 1, 2$

~~$B_2$~~   $\rightarrow 3$

$B_3 \rightarrow 4$

$B_4 \rightarrow 5, 6, 7, 8$

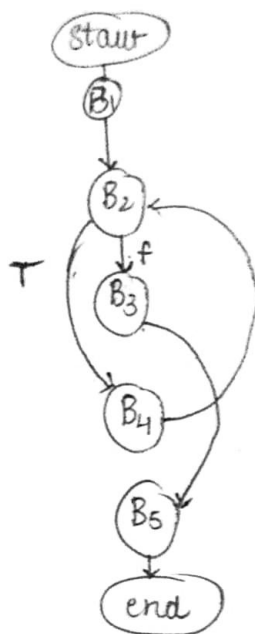
$B_5 \rightarrow 9$

# Basic Block = 5

# Control = 5

# nodes = 7

# edges = 7



Eg  $\rightarrow 2$ .

$a = 10$

$b = 20$

while ( $a + b > c$ )

{  $c = c + 1$ ;

}

$a = b * d$ ;

1.  $a = 10 \rightarrow \text{leader}$

2.  $b = 20$

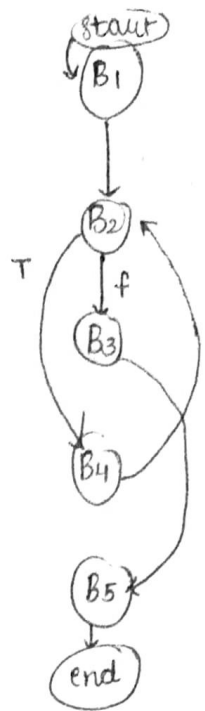
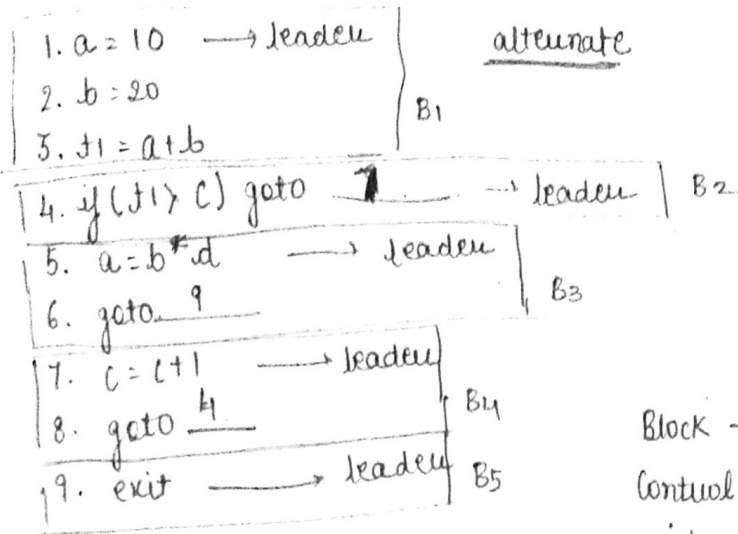
3. ~~if ( $a + b > c$ )~~  $t_1 = a + b$

4. if ( $t_1 > c$ ) goto T.  $\rightarrow \text{leader}$

5.  $c = c + 1 \rightarrow \text{leader}$

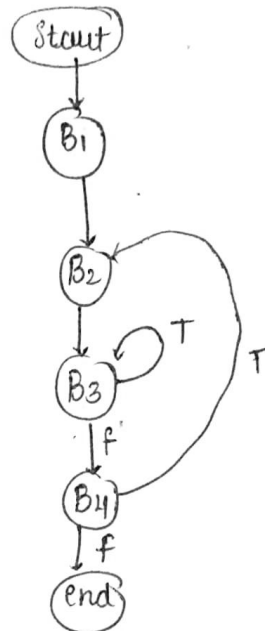
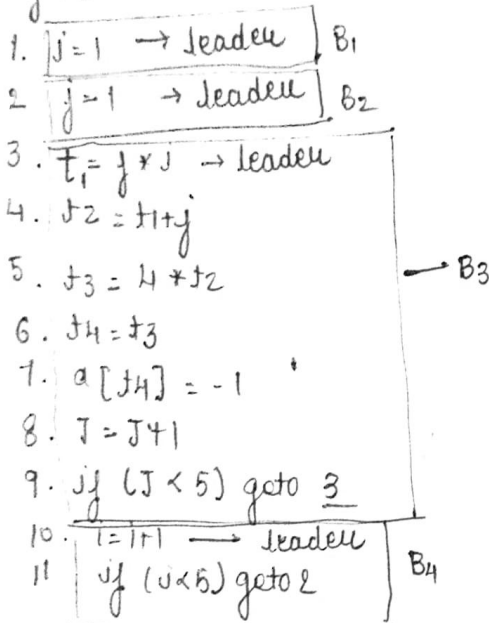
6. goto 4.

7.  $a = b * d \rightarrow \text{leader}$



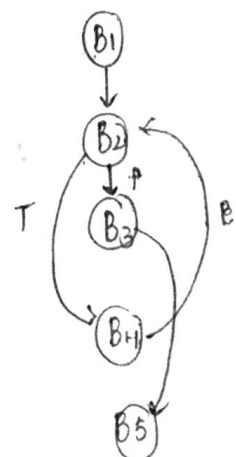
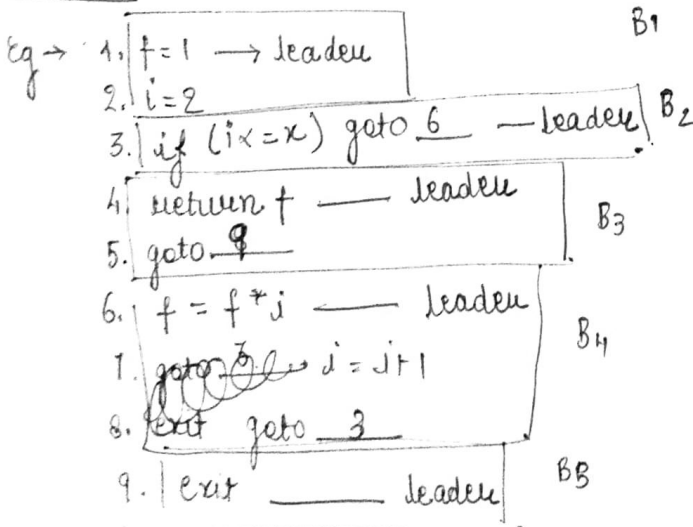
Block - 5  
 Control - 5  
 node - 7  
 edge - 7

eg  $\rightarrow 3$

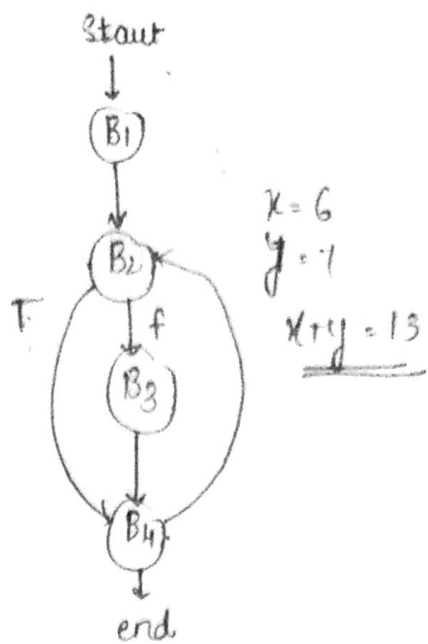


nodes - 6  
 edges - 7

### Lecture 31

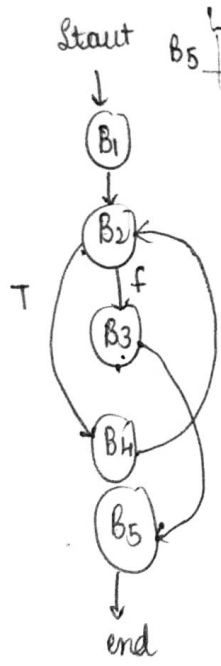


Que. 1.  $i = 2$  — leader  $B_1$   
 2.  $J = 1$   
 3.  $a = i * j$  — leader  $B_2$   
 4.  $b = i + j$   
 5.  $\text{if}(b < a) \text{ goto } 7$   
 6.  $a = a + 1$  — leader  $B_3$   
 7.  $j = j + 1$  — leader  $B_4$   
 8.  $\text{goto } 3$

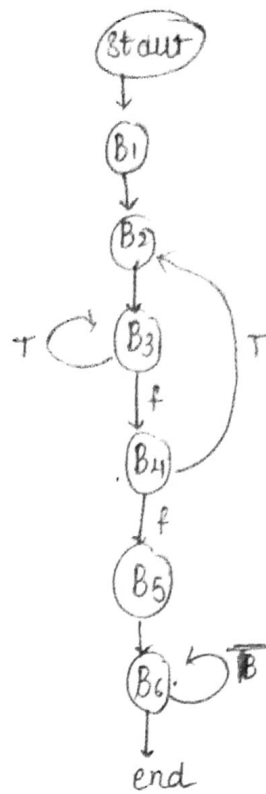


Que  $J = 1$   
 while  $(J \leq n)$   
 {  
 $R = a + b;$   
 $R = R * c;$   
 $R = R + d$   
 $J++$   
 }

$B_1$  1.  $J = 1$  — leader  
 $B_2$  2.  $\text{if}(J \leq n) \text{ goto } 4$  — leader  
 $B_3$  3.  $\text{goto } 9$  — leader  
 $B_4$  4.  $R = a + b$  — leader  
 5.  $R = R * c$   
 6.  $R = R + d$   
 7.  $J = J + 1;$   
 8.  $\text{goto } 2$   
 $B_5$  9.  $\text{exit}$  — leader



|                                           |        |                |
|-------------------------------------------|--------|----------------|
| 1. $j = 1$                                | leader | B <sub>1</sub> |
| 2. $j = 1$                                | leader | B <sub>2</sub> |
| 3. $t_1 = 10 * j$                         | leader | B <sub>3</sub> |
| 4. $t_2 = j + j$                          |        |                |
| 5. $t_1 = 7 * t_2$                        |        |                |
| 6. $t_4 = t_3 - 88$                       |        |                |
| 7. $a[t_4] = 0$                           |        |                |
| 8. $i = j + 1$                            |        |                |
| 9. $\text{if } j < 10 \text{ goto } 3.$   |        |                |
| 10. $j = j + 1$                           | leader | B <sub>4</sub> |
| 11. $\text{if } j < 10 \text{ goto } 2$   |        | B <sub>5</sub> |
| 12. $j = 1$                               | leader |                |
| 13. $t_5 = j - 1$                         | leader | B <sub>6</sub> |
| 14. $t_6 = 88 * t_5$                      |        |                |
| 15. $a[t_6] = 1$                          |        |                |
| 16. $j = j + 1$                           |        |                |
| 17. $\text{if } j < 10 \text{ goto } 13.$ |        |                |



node - 8  
edge - 10

Que  $X = A/b$   
 $Y = C + D$   
 $Y = Y - X$   
 $X = D + Y$   
 $Z = f + Y$   
 $Z = Z + 4$

min raw. req to convert to SSA

$X_1 = A/b$   
 $Y_1 = C + D$   
 $Y_2 = Y_1 - X_1$   
 $X_2 = D + Y_2$   
 $Z_1 = f + Y_2$   
 $Z_2 = Z_1 + 4$

11

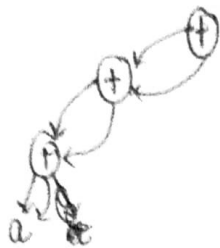
PYQ

Ans 1.  $\text{if } (x < y) \text{ goto } 3$   
 2. goto 12.  
 3.  $\text{if } (u < v) \text{ goto } 5.$   
 4. goto 12.  
 5.  $\text{if } (x = 1) \text{ goto } 10$   
 6.  $\text{if } (x <= v) \text{ goto } 8.$   
 7. goto 11.

8.  $x = x + 3$   
 9. goto 6.  
 10.  $y = y + 1$   
 11. goto 1.  
 12. exit

$$(((a+a) + (a+a)) + ((a+a) + (a+a)))$$

using rule. + is left to right associative



Boolean Expression Reduction

1.  $B \rightarrow B_1 \parallel B_2$  (logical OR)

$$B_1.\text{true} = B.\text{true}$$

$$B_1.\text{false} = \text{newlabel}()$$

$$B_2.\text{true} = B.\text{true}$$

$$B_2.\text{false} = B.\text{false}$$

$$B.\text{code} = B_1.\text{code} \parallel \text{label}(B_1.\text{false}) \parallel B_2.\text{code}$$

2.  $B \rightarrow B_1 \& B_2$  (logical And)

$$B_1.\text{true} = \text{newlabel}()$$

$$B_1.\text{false} = B.\text{false}$$

$$B_2.\text{true} = B.\text{true}$$

$$B_2.\text{false} = B.\text{false}$$

$$B.\text{code} = B_1.\text{code} \parallel \text{label}(B_1.\text{true}) \parallel B_2.\text{code}$$

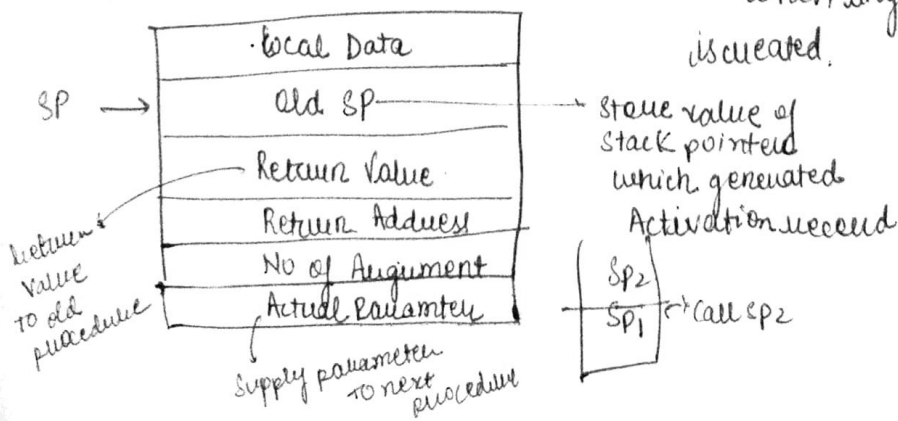
3.  $B \rightarrow ! B_1$  (NOT)

$$B_1.\text{false} = B.\text{true}$$

$$B_1.\text{true} = B.\text{false}$$

$$B.\text{code} = B_1.\text{code}$$

Activation Record - data structure which is invoked when any procedure is created.



## 5 Error Recovery Strategies

- automatic type conversion is done by compiler
- ① Panic mode - pause - discard i/p symbol one at a time
  - ② Symbol table - pause - perform original connection like missing ;
  - ③ Global Connection
  - ④ Phrase level recovery
  - ⑤ Error production → specific errors are there with grammar & if error occurs specific message is displayed
- 3 ET  
 changes  
 42 AT  
 1120 TT  
 i/p string  
 me  
 am finally least amount  
 of changes AT GT TT 4221 E